Assignment #2: Vole Machine

Course Code: CS213

Course Instructor: Dr. Mohammad El-Ramly

Submission Date: 9-11-2026

Academic Year: 2026–2027

Team Contact Number: +20 112842298

Team Members:

- Student #1 [Team Leader] :
    - Name: Omar Wael Omar Abd Al-Latif
    - ID: 20250444
    - Email: 20250444@stud.fci-cu.edu.eg

- Student #2:
    - Name: Kirolos Nabil Ataa Allah
    - ID: 20250503
    - Email: 20250503@stud.fci-cu.edu.eg

- Student #3:
    - Name: Seif Sabry Abd Al-Salam
    - ID: 20250315
    - Email: 20250315@stud.fci-cu.edu.eg

- Student #4:
    - Name: Mohamed Ahmed Mohamed Mostafa
    - ID: 20250538
    - Email: 20250538@stud.fci-cu.edu.eg

Breakdown (who did what):

- Omar Wael Omar Abd Al-Latif
  - Contributing to the initial design, the other three designs and picking the final design.
  - Contributing to make phase 0’s file.

- Kirolos Nabil Ataa Allah
  - Contributing to the initial design, the other three designs and picking the final design.
  - Contributing to make phase 0’s file.

- Seif Sabry Abd Al-Salam
  - Contributing to the initial design, the other three designs.
  - Contributing to make phase 0’s video script.

- Mohamed Ahmed Mohamed Mostafa
  - Contributing to the initial design, the other three designs and picking the final design.
  - Contributing to recording phase 0’s video.

1. Introduction

The Vole Machine is a virtual, educational computer architecture designed to model the fundamental concepts of computer organization, machine language execution, and CPU operations. It serves as an ideal pedagogical model to understand how hardware components interact at a low level.

Core Architectural Components:

1. Main Memory: The machine features a main memory consisting of 256 addressable cells, spanning from addresses `00` to `FF` in hexadecimal. Address `FF` is a memory-mapped I/O location dedicated to the screen, meaning any write operation to address `FF` directly outputs data to the screen display.

2. Registers: The CPU contains 16 general-purpose registers labeled from `R0` to `RF`, each storing an 8-bit (1-byte) bit pattern.

3. Control Unit & Registers:
    - Program Counter (PC): Holds the memory address of the next instruction to be fetched and executed.
    - Instruction Register (IR): Holds the 16-bit (2-byte) instruction currently being fetched and decoded.

4. Instruction Cycle (Fetch-Decode-Execute): The Vole machine operates by continuously performing a cycle where instructions are fetched from memory into the IR (advancing the PC by 2 bytes), decoded into their constituent operation codes and operands, and finally executed by the CPU. Supported operations include data transfer (load/store/move), arithmetic operations (two's complement and floating-point addition), logical operations (OR, AND, XOR), bitwise rotations, conditional jumps, and a HALT instruction to terminate execution.

2. Initial Design

2.1 Design Overview

Our initial design models the Vole machine as a set of cooperating classes. The Simulator class is the entry point and handles user interaction. It owns a Machine, which represents the computer itself, and a Screen for output. The Machine aggregates the main components: CPU, Memory, Registers, and ProgramLoader. The CPU performs the fetch-decode-execute cycle using the PC, IR, and Decoder classes, and represents each decoded instruction as an Instruction object. Writing to memory address FF is forwarded to the Screen, as the memory-mapped output the assignment requires.

2.2 Classes and Responsibilities

Class	Responsibility
Simulator	Entry point. Shows the main menu, drives the user interaction, and resets the system.
Machine	Represents the whole Vole computer. Groups the CPU, memory, registers and loader, and coordinates loading and running a program.
ProgramLoader	Reads a program file, ignores comments and empty lines, validates the instructions, and returns the program as a list of instruction words.
CPU	Executes instructions by running the fetch-decode-execute cycle. Holds the PC, IR and decoder, and references Memory and Registers.
PC	Program Counter. Stores the address of the next instruction.
IR	Instruction Register. Stores the 16-bit instruction being executed.
Decoder	Converts a raw 16-bit instruction into an Instruction object.

Instruction	Represents a decoded instruction as four 4-bit fields: opcode, R, X, Y.
Memory	256 cells (addresses 00 to FF), each 8 bits. Address FF is memory-mapped to the screen.
Registers	16 general-purpose registers (R0 to RF), each 8 bits.
Screen	Stores and displays the program output written to address FF.

2.3 Attributes and Methods

Simulator

- Attributes: - machine : Machine, - screen : Screen
- Methods: + run() : void, + show_main_menu() : void, + reset() : void

Machine

- Attributes: - cpu : CPU, - memory : Memory, - registers : Registers, - loader : ProgramLoader
- Methods: + load_program(start : uint8_t, file : string) : void, + run_program(counter : int) : void

ProgramLoader

- Methods: + load_program(file : string) : vector<uint16_t>, + validate_program(program : vector<uint16_t>) : bool

CPU

- Attributes: - pc : PC, - ir : IR, - memory : Memory&, - registers : Registers&, - decoder : Decoder, - decoded_instruction : Instruction
- Methods: + run_instruction() : void

PC

- Attributes: - value : uint8_t

- Methods: + get() : uint8_t, + set(value : uint8_t) : void, + increment(amount : uint8_t) : void

IR

- Attributes: - value : uint16_t
- Methods: + get() : uint16_t, + set(value : uint16_t) : void

Decoder

- Methods: + decode(instruction : uint16_t) : Instruction

Instruction

- Attributes: - opcode : uint8_t (4-bit), - r : uint8_t (4-bit), - x : uint8_t (4-bit), - y : uint8_t (4-bit)
- Methods: + get_opcode(), + get_r(), + get_x(), + get_y(), each returning uint8_t

Memory

- Attributes: - cells : vector<uint8_t>, - screen : Screen&
- Methods: + write(val : uint8_t, address : uint8_t) : void, + read(address : uint8_t) : uint8_t, + reset() : void

Registers

- Attributes: - vals : vector<uint8_t>
- Methods: + read(address : uint8_t) : uint8_t, + set(val : uint8_t, address : uint8_t) : void, + reset() : void

Screen

- Attributes: - output : string
- Methods: + write(val : uint8_t) : void, + display() : void, + reset() : void

2.4 Relationships

- Simulator ◆ Machine and Simulator ◆ Screen (composition): the simulator owns the machine and the screen.
- Machine ◆ CPU, Memory, Registers, ProgramLoader (composition): the machine is made of these components.

- CPU ◆ PC, IR, Decoder (composition): each CPU has its own program counter, instruction register and decoder.
- CPU ◆ Instruction (composition, 0..1): the CPU holds the currently decoded instruction, if any.
- CPU → Memory and CPU → Registers (association): the CPU accesses memory and registers through references.
- Decoder ⇢ Instruction (dependency, «creates»): the decoder creates Instruction objects.
- Memory → Screen (association): a write to address FF is forwarded to the screen.

2.5 PlantUML Code :

```
@startuml
title Vole Machine Simulator - UML Class Diagram

class Simulator {
  - machine : Machine
  - screen : Screen
  + run() : void
  + show_main_menu() : void
  + reset() : void
}

class Machine {
  - cpu : CPU
  - memory : Memory
  - registers : Registers
  - loader : ProgramLoader
  + load_program(start : uint8_t, file : string) : void
  + run_program(counter : int) : void
}

class ProgramLoader {
  + load_program(file : string) : vector<uint16_t>
  + validate_program(program : vector<uint16_t>) : bool
}

class CPU {
  - pc : PC
```

```
  - ir : IR
  - memory : Memory&
  - registers : Registers&
  - decoder : Decoder
  - decoded_instruction : Instruction
  + run_instruction() : void
}

class PC {
  - value : uint8_t
  + get() : uint8_t
  + set(value : uint8_t) : void
  + increment(amount : uint8_t) : void
}

class IR {
  - value : uint16_t
  + get() : uint16_t
  + set(value : uint16_t) : void
}

class Decoder {
  + decode(instruction : uint16_t) : Instruction
}

class Instruction {
  - opcode : uint8_t <<4-bit>>
  - r : uint8_t <<4-bit>>
  - x : uint8_t <<4-bit>>
  - y : uint8_t <<4-bit>>
  + get_opcode() : uint8_t
  + get_r() : uint8_t
  + get_x() : uint8_t
  + get_y() : uint8_t
}

class Memory {
  - cells : vector<uint8_t>
  - screen : Screen&
```

```
  + write(val : uint8_t, address : uint8_t) : void
  + read(address : uint8_t) : uint8_t
  + reset() : void
}

class Registers {
  - vals : vector<uint8_t>
  + read(address : uint8_t) : uint8_t
  + set(val : uint8_t, address : uint8_t) : void
  + reset() : void
}

class Screen {
  - output : string
  + write(val : uint8_t) : void
  + display() : void
  + reset() : void
}

note right of Memory
  256 cells, addresses 00-FF.
  Address FF is memory-mapped
  to the screen.
end note

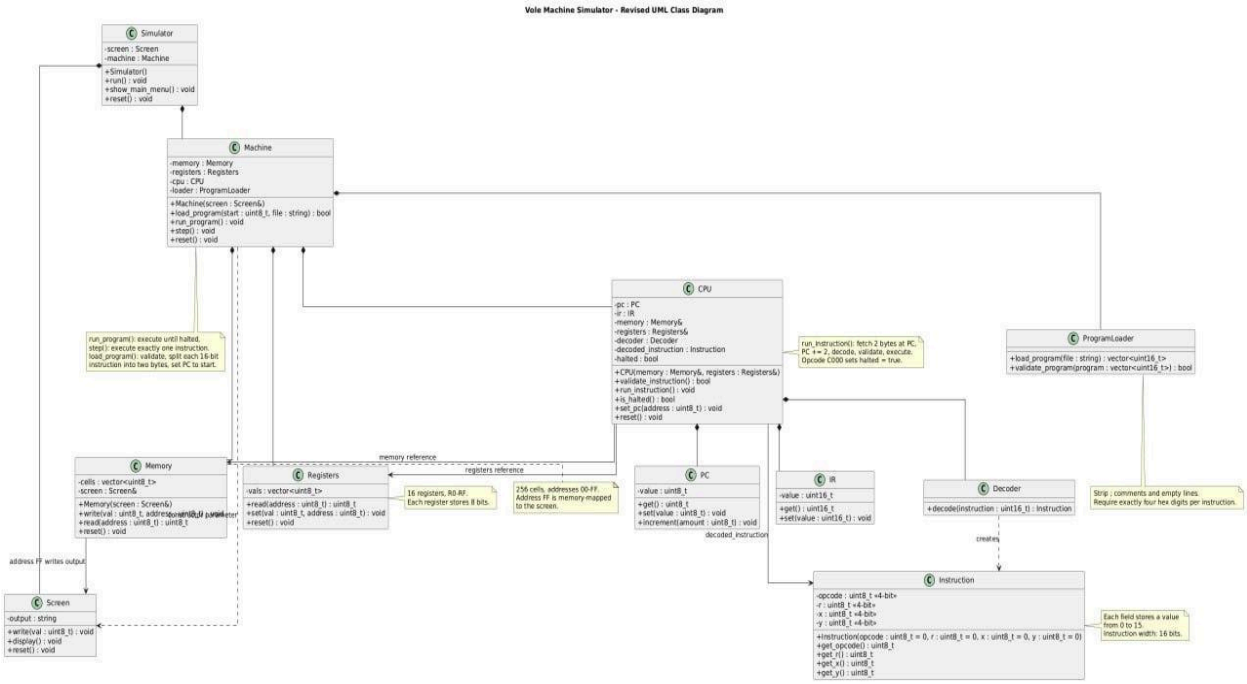
note right of Registers
  16 registers, R0-RF.
  Each register stores 8 bits.
end note

note right of Instruction
  Each field stores a value
  from 0 to 15.
  Instruction width: 16 bits.
end note

Simulator *-- Machine
Simulator *-- Screen
Machine *-- CPU
```

```
Machine *-- Memory
Machine *-- Registers
Machine *-- ProgramLoader
CPU *-- PC
CPU *-- IR
CPU *-- Decoder
CPU *-- "0..1" Instruction : decoded_instruction
CPU --> Memory : memory reference
CPU --> Registers : registers reference
Decoder ..> Instruction : creates
Memory --> Screen : address FF writes output
@enduml
```

2.6 UML Class Diagram



3. AI Designs

3.0 Overview

To evaluate our UML class diagram for the Vole Machine Simulator, we used three AI assistants: Gemini, ChatGPT, and Claude. The same standardized prompt and initial design were provided to each model in a separate conversation.

The models reviewed the design based on object-oriented principles, C++ initialization and memory safety, program loading, instruction validation, and execution control. Our goal was to identify necessary improvements while preserving the original architecture and avoiding unnecessary complexity.

After obtaining the three designs, we submitted them to a fifth AI model, Qwen, to compare the alternatives and select the strongest candidate for the final architecture.

The complete shared prompt and the full AI conversations, including their PlantUML code, are documented in the appendices. The following sections summarize the three designs, present their diagrams, and compare their main contributions.

3.0.1 Common Prompt

The same prompt was submitted independently to Gemini, ChatGPT, and Claude. It requested a brief assessment, minimal necessary changes, a complete revised PlantUML diagram, a change summary, and an implementation difficulty assessment.

The complete prompt is provided in Appendix A.

3.1 Model 1: Gemini

3.1.1 Design Overview

Gemini focused on clarifying hardware dependencies and supporting single-step execution. It proposed connecting `Screen` to `Memory`, adding `Machine::step()`, and introducing `CPU::validate_instruction()`.

It also removed the unnecessary counter parameter from `Machine::run_program()`, allowing the CPU's program counter to manage execution progress.

3.1.2 UML Class Diagram

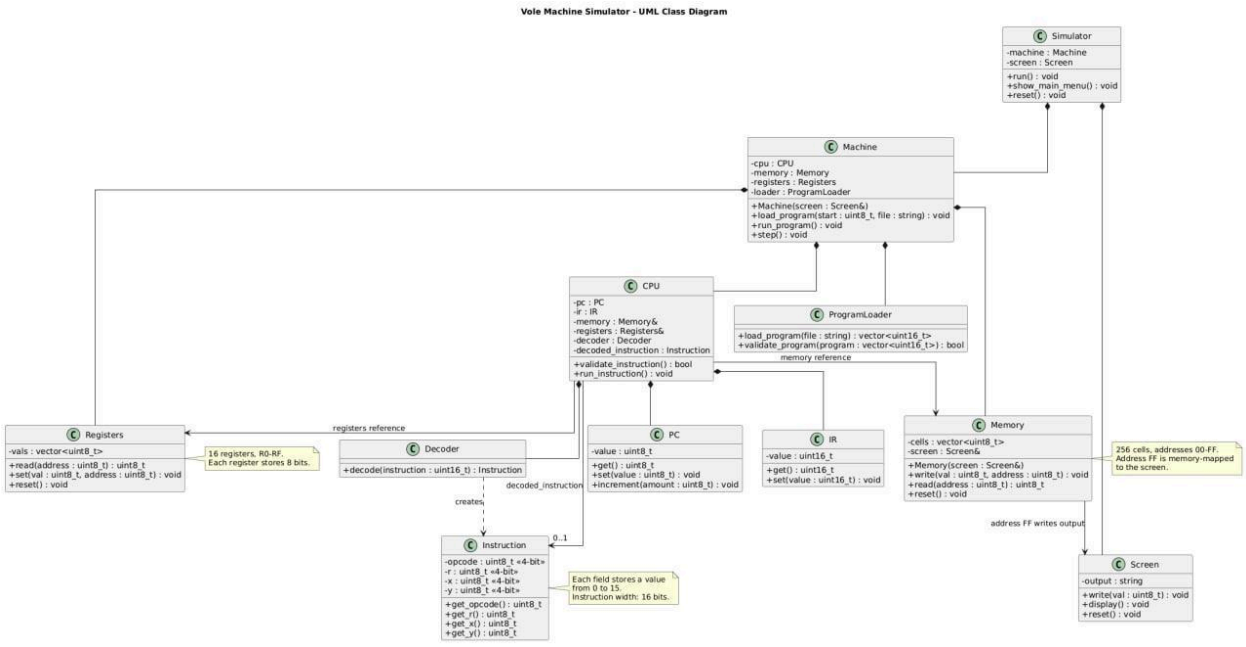


Figure 1: UML Class Diagram Proposed by Gemini.

3.1.3 Analysis

Strengths: Gemini preserved the original class structure and proposed relatively simple changes to hardware connectivity, instruction validation, and single-step execution.

Limitations: Its diagram did not include an explicit HALT-state mechanism. It also left program-loading failure reporting and some initialization requirements unresolved.

The complete Gemini conversation and its PlantUML code are provided in Appendix B.

3.2 Model 2: ChatGPT

3.2.1 Design Overview

ChatGPT focused on memory boundaries, class relationships, and the separation of program validation from instruction validation.

It proposed renaming Decoder to InstructionDecoder, representing the relationship between CPU and Instruction as composition, and clarifying the responsibilities of the program loader and CPU.

It also highlighted the importance of address FF, which is reserved for screen output, and the need to prevent instruction fetching or program loading from violating memory boundaries.

3.2.2 UML Class Diagram

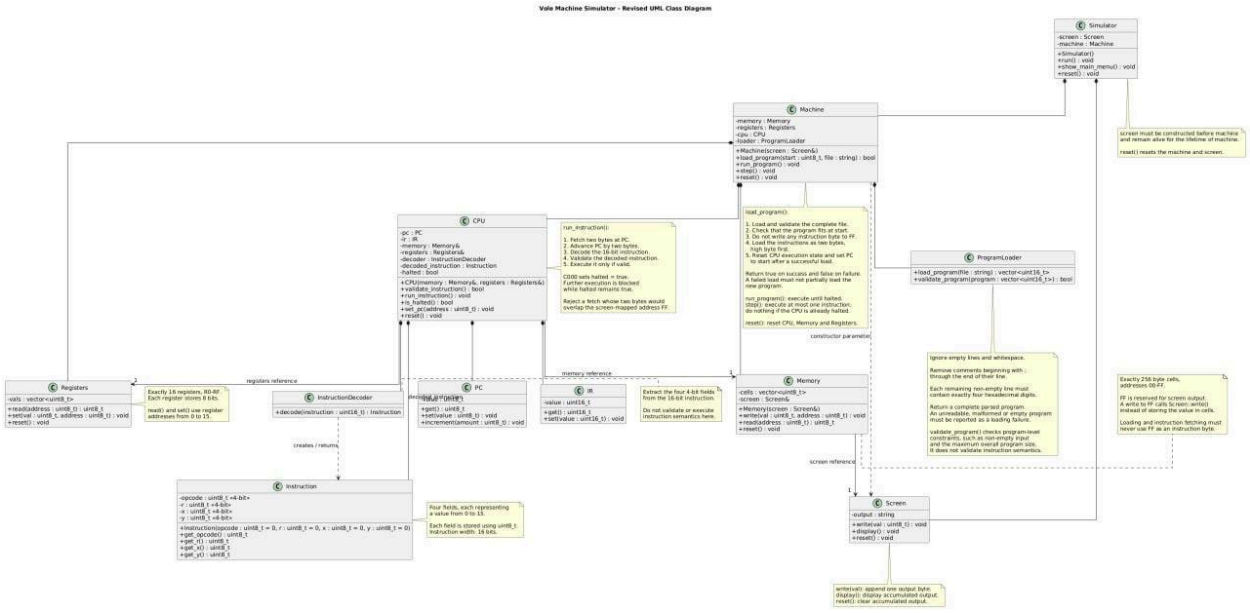


Figure 2: UML Class Diagram Proposed by ChatGPT.

3.2.3 Analysis

Strengths: ChatGPT provided detailed explanations of object relationships, memory-boundary restrictions, and

instruction-validation responsibilities. It also retained support for single-step execution and CPU halt-state management.

Limitations: The program-loading interface remained largely unchanged, and the design did not add an explicit constructor to initialize the register container. Some recommendations were specified in notes rather than incorporated into class interfaces.

The complete ChatGPT conversation and its PlantUML code are provided in Appendix C.

3.3 Model 3: Claude

3.3.1 Design Overview

Claude provided a more extensive review of initialization, program-loading interfaces, and execution-state consistency.

It identified that validating the four-hexadecimal-digit input format requires access to the original file contents. It therefore proposed changing the loader interface to return a Boolean result and provide the parsed instructions through a reference parameter.

Claude also proposed adding a `Registers()` constructor, initializing `PC`, `IR`, and the CPU halt flag, and resetting CPU execution state after successfully loading a new program.

3.3.2 UML Class Diagram

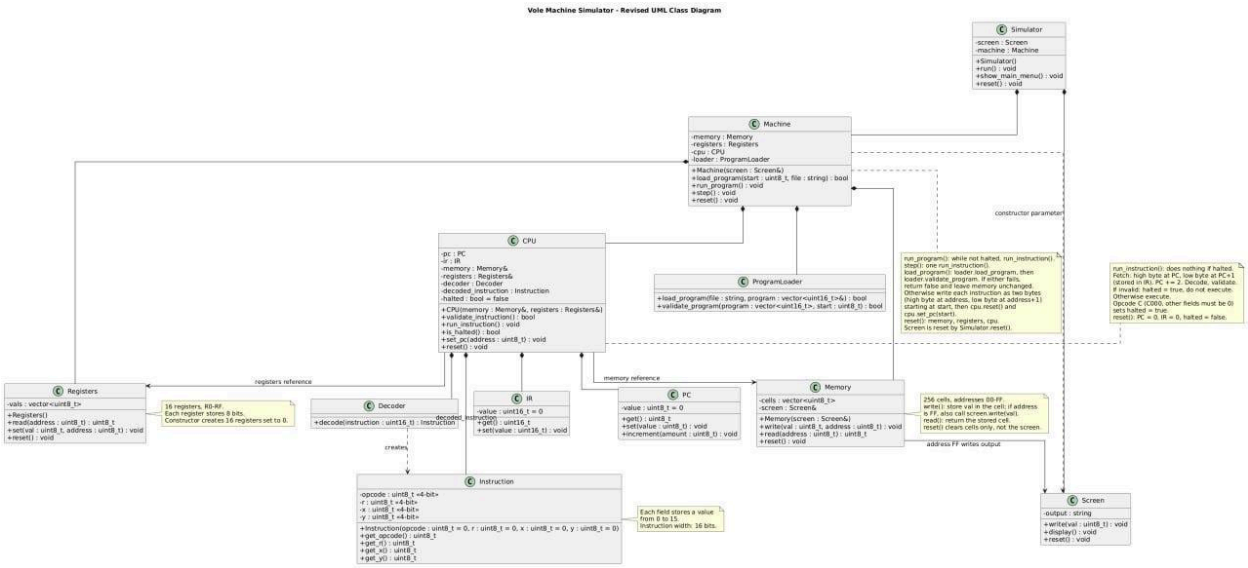


Figure 3: UML Class Diagram Proposed by Claude.

3.3.3 Analysis

Strengths: Claude addressed concrete interface and initialization problems while preserving the original class structure. Its proposed changes provided a stronger foundation for program loading and execution-state management.

Limitations: The design still required verification against the complete instruction set and assignment requirements, particularly invalid-instruction handling, memory boundaries, and screen-output behavior.

The complete Claude conversation and its PlantUML code are provided in Appendix D.

3.4 Qwen: Comparative Evaluation

3.4.1 Purpose

After obtaining the three designs, we submitted them to Qwen for an independent comparative evaluation. Qwen was asked to compare their

strengths and weaknesses and identify the most suitable candidate for the final architecture.

3.4.2 Qwen Conversation

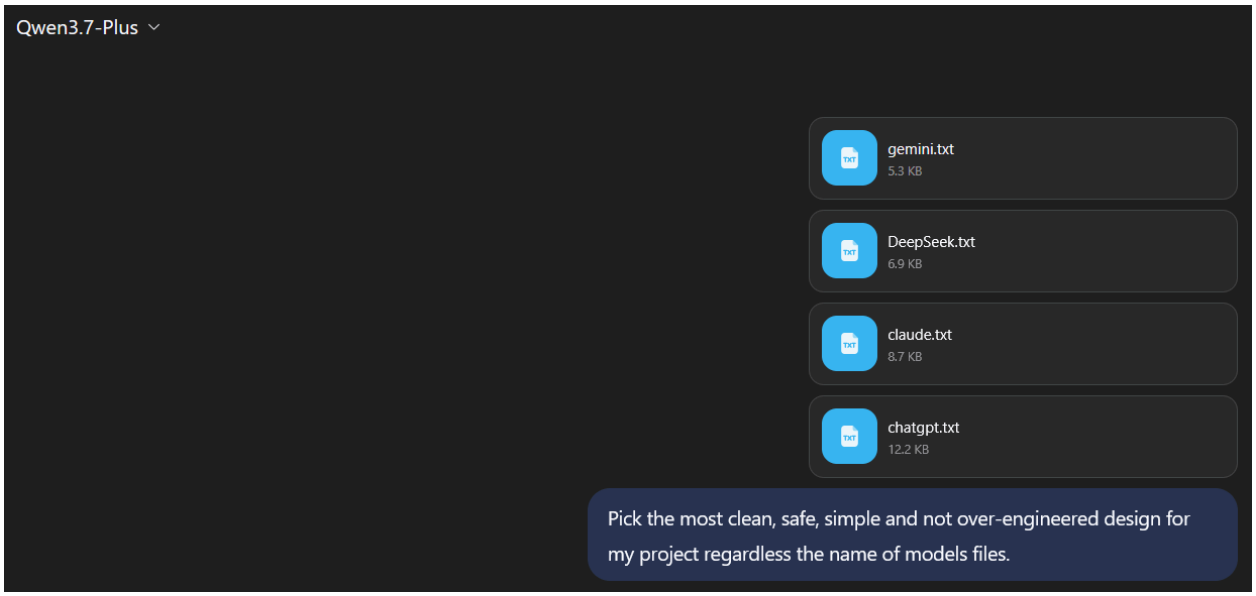


Figure 4: Qwen's Comparative Evaluation of the Three AI-Generated Designs.

Based on your criteria of being **clean, safe, simple, and not over-engineered**, the best design is the one from **`claude.txt`**.

Here is a detailed breakdown of why this design wins, along with why the others fall short in specific areas:

Why `claude.txt` is the Best Choice

1. It is the most "Safe" (C++ Correctness & Error Handling)

* **Prevents Uninitialized Memory:** It is the only design that explicitly catches that `PC.value`, `IR.value`, and `CPU.halted` lack default

initializers. In C++, leaving a `bool` or integer uninitialized can cause the simulator to randomly halt or crash before it even starts.

* **Fixes the Empty Vector Trap:** It correctly identifies that `Registers` needs a constructor to initialize the `std::vector` to a size of 16. Without this, the very first register access would result in an out-of-bounds memory error.

* **Safe Error Reporting without Exceptions:** It changes `ProgramLoader::load_program` to take a `vector<uint16_t>&` (pass-by-reference) and return a `bool`. This allows the loader to safely report file/format errors to the `Machine` without forcing a first-year student to learn and implement C++ exception handling (`try/catch`) or `std::optional`.

* **Bounds Checking:** It adds the `start` address to `validate_program()`, ensuring the program actually fits in memory before a single byte is written, preventing silent memory corruption.

2. It is the most "Clean" (Clear Intent & Proper UML)

* **Proper Ownership:** It correctly changes the relationship between `CPU` and `Instruction` from a standard association (`-->`) to composition (`*--`), accurately reflecting that `decoded_instruction` is a value member owned by the CPU.

* **Single-Responsibility Methods:** It keeps `step()` and `run_program()` as separate methods. This is much cleaner than using a "magic number" parameter to control execution flow.

3. It is "Simple and Not Over-Engineered"

* **No Feature Creep:** It explicitly lists tempting but unnecessary additions (like renaming `Decoder` to `InstructionDecoder`, adding read-only getters, or creating a separate error-reporting class) and consciously **excludes** them from the main diagram.

* **Beginner-Friendly:** It relies entirely on fundamental C++ concepts (constructors, default initializers, pass-by-reference, and composition) without introducing advanced templates or design patterns.

Why the other designs were rejected:

* **`gemini.txt` (Unsafe):** It leaves `Machine::load_program()` returning `void`. If a file is missing or malformed, the `Machine` has no way to know, meaning it will either crash or execute garbage data. Returning a `bool` is strictly necessary for safety.

* **`DeepSeek.txt` (Unclean):** It uses `run_program(counter : int)` where `counter = 1` means "step" and `counter < 0` means "run until halt". Using an integer parameter to dictate control-flow logic is a code smell and makes the API confusing and unclean compared to having a dedicated `step()` method.

* **`chatgpt.txt` (Over-complicated Error Handling):** While it has excellent behavioral notes, its UML keeps `ProgramLoader::load_program()` returning a `vector<uint16_t>`. Because it returns a vector, it has no safe, simple way to report parsing errors (like a malformed hex string) without throwing exceptions or using `std::optional`, which adds unnecessary complexity for a first-year project.

Conclusion: Go with the **`claude.txt`** design. It anticipates the exact C++ pitfalls a first-year student will face (uninitialized variables, empty vectors, silent file errors) and patches them using the simplest, most standard C++ tools available.

3.4.3 Evaluation Outcome

Qwen selected Claude's design as the strongest candidate among the three alternatives. We therefore selected Claude's design as the basis for the final architecture.

The selection emphasized the proposed improvements to program loading, register initialization, default execution state, and reset behavior. The chosen design was then considered for use as the final architecture, subject to verification against the assignment requirements.

3.5 Comparison of the Three AI Designs

The following table summarizes the main differences between the three designs.

Criterion	Gemini	ChatGPT	Claude
HALT-state handling	No explicit halt mechanism	Includes halt-state support	Includes halt-state support with a default value
Single-step execution	Added `step()`	Added `step()`	Added `step()`
Program-loading interface	Required further refinement	Largely unchanged	Added explicit parsing and validation interfaces
Register initialization	No explicit constructor	No explicit constructor	Added `Registers()`
Primitive state initialization	Required further refinement	Required further refinement	Added explicit defaults

CPU–Instruction relationship	Association	Composition	Composition
Decoder name	`Decoder`	`InstructionDecoder`	`Decoder`
Main contribution	Hardware wiring and execution interfaces	Relationships and memory-boundary rules	Interface and initialization improvements

The comparison shows that the models emphasized different aspects of the design. Gemini concentrated on hardware connectivity and execution interfaces, ChatGPT clarified object relationships and memory-boundary rules, and Claude proposed more extensive changes to loading and initialization.

Qwen subsequently selected Claude's design as the strongest candidate, providing an additional stage of comparative evaluation before the final architecture was selected.

---

4. Final Design

4.1 Selected Design

Following Qwen's comparative evaluation, Claude's design was selected as the basis for the final architecture.

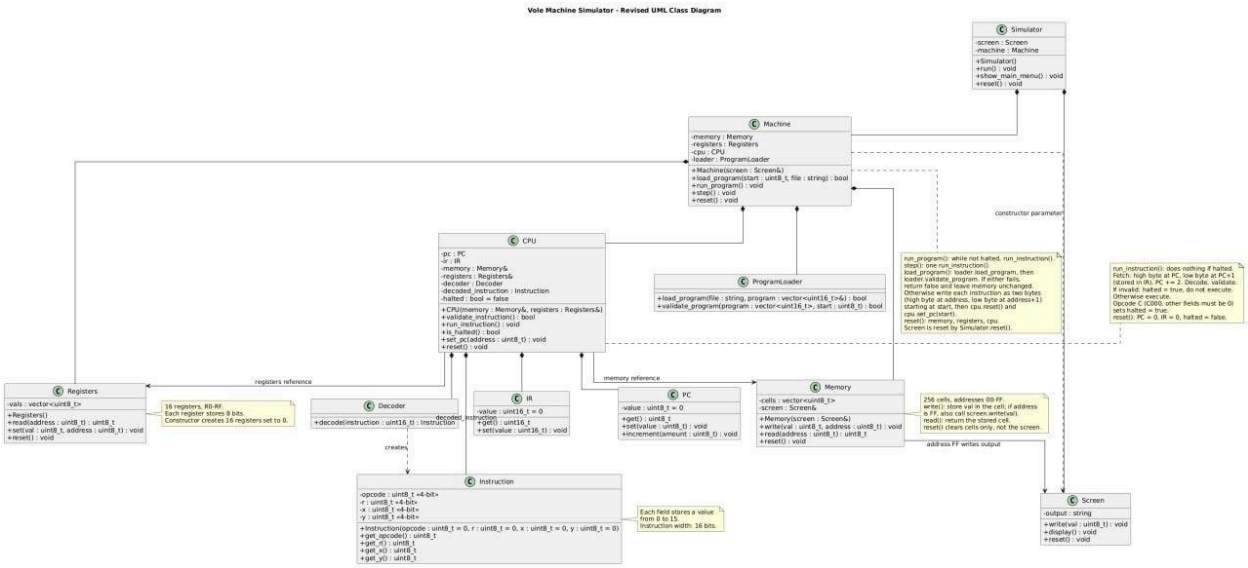


Figure 5: Final UML Class Diagram Based on Claude's Selected Design.

The final diagram should represent the selected architecture. If no changes were made after Claude's design was selected, use the same diagram as Figure 3 without duplicating its PlantUML code in this section.

4.2 Final Architecture and Responsibilities

Simulator: Controls the application flow, displays the main menu, and coordinates the Screen and Machine objects.

Machine: Coordinates memory, registers, CPU execution, program loading, single-step execution, and machine reset operations.

CPU: Manages instruction fetching, decoding, validation, execution, program-counter updates, and the HALT state.

Memory and Registers: Represent the machine's 256 addressable memory cells and 16 general-purpose registers.

ProgramLoader: Reads program files, ignores comments and empty lines, checks hexadecimal instruction formatting, and validates programs before loading.

Screen: Buffers and displays output produced through the memory-mapped screen address FF.

PC, IR, Decoder, and Instruction: Support instruction addressing, instruction storage, instruction decoding, and access to decoded instruction fields.

4.3 Design Verification

Before implementation, the selected architecture must be checked against the assignment requirements. The main verification points are:

1. Support for all required instructions, including C000 for HALT and 3RFF for screen output.
2. Rejection of invalid instructions and invalid programs.
3. Correct handling of starting addresses, even-address requirements, and memory boundaries.
4. Proper initialization and reset behavior.
5. Correct continuous and single-step execution.
6. Consistent handling of writes to the memory-mapped screen address.

Any changes made during subsequent refinement must be reflected in the final PlantUML diagram.

4.4 Why Claude's Design Was Selected?

Claude's design was selected because it addressed several concrete interface and initialization problems while preserving the original architecture.

Its proposed program-loading interface communicates success or failure explicitly. The Registers() constructor provides a clear initialization point, while default values for primitive state attributes help establish a valid initial state. Its reset approach also helps prevent a previously halted CPU from blocking a newly loaded program.

Qwen's comparative evaluation supported selecting Claude's design as the strongest starting point among the three alternatives. However,

the design must still be verified against the complete assignment specification before implementation.

4.5 What did we Learn?

The comparison showed that different AI assistants can identify different weaknesses in the same design. Some focused on hardware connectivity and object relationships, while others emphasized execution control, initialization, and program loading.

We learned that AI-generated designs must be evaluated critically rather than accepted automatically. Each proposed change must be checked for correctness, necessity, and compatibility with the assignment requirements.

Comparing the three alternatives and using Qwen as an additional evaluator helped us identify a stronger foundation for the final architecture.

Appendices

Appendix A: Common Prompt

The following prompt was submitted independently to Gemini, ChatGPT, and Claude.

I have an initial UML class design for a Vole Machine Simulator written in C++. I want you to review and improve the existing design with minimal necessary changes only.

Main Goal

Make the design correct, understandable, and straightforward to implement for a first-year Computer Science student using basic C++ OOP.

Do not over-engineer the project. Preserve the original design as much as possible.

Rules

Keep the existing classes, responsibilities, and relationships unless a change is genuinely necessary.

Do not add design patterns, unnecessary abstraction layers, inheritance, interfaces, factories, or extra classes just to make the design look more sophisticated.

Do not split a class into multiple classes unless there is a clear correctness or responsibility problem.

Do not add methods or attributes unless they are required by the simulator's functionality or fix a real design issue.

Keep the design easy to implement using ordinary C++ classes, constructors, references, and simple methods.

Preserve the existing InstructionDecoder and Instruction classes. The decoder should extract instruction fields; the CPU should validate and execute instructions.

Preserve the decision that Simulator owns Machine and Screen, while Memory holds a reference to the same Screen object to handle writes to address FF.

Avoid duplicating responsibilities between classes. In particular, keep instruction execution in the CPU, instruction decoding in the decoder, program loading in the loader, and screen output in the screen class.

Do not write implementation code. This task is only about reviewing and updating the UML class design.

Do not redesign the project just because another architecture is possible. Prefer the smallest reasonable change to the current design.

Review Requirements

Check the design for:

Missing or incorrect class relationships and ownership.

Incorrect or duplicated responsibilities.

Missing essential attributes or methods.

Type correctness for the 256-byte memory, 16 registers, 8-bit addresses, and 16-bit instructions.

Correct handling of the memory-mapped screen at address FF.

A clear fetch-decode-validate-execute cycle.

Support for running the whole program and executing one instruction at a time.

Program loading requirements, including comments beginning with ;, empty lines, and four-hex-digit instructions.

Correct handling of the halt instruction C000.

Do not add features that are not required by these responsibilities.

Required Output

Brief assessment: List only genuine problems in the current design.

Minimal changes: For each proposed change, explain why it is necessary. Clearly distinguish necessary fixes from optional improvements.

Updated UML: Provide the complete revised class diagram in PlantUML.

Change summary: List exactly what changed from the initial design and what remained unchanged.

Implementation difficulty: Briefly explain whether the revised design is still suitable for a beginner to implement in C++.

If a proposed change is optional rather than necessary, leave it out of the main design and mention it separately.

Here is my initial design:

```
@startuml

title Vole Machine Simulator - Revised UML Class Diagram

skinparam classAttributeIconSize 0

skinparam linetype ortho


class Simulator {

    - screen : Screen

    - machine : Machine

    + Simulator()

    + run() : void

    + show_main_menu() : void

    + reset() : void

}


class Machine {

    - memory : Memory

    - registers : Registers

    - cpu : CPU

    - loader : ProgramLoader

    + Machine(screen : Screen&)
```

```
    + load_program(start : uint8_t, file : string) : bool
    + run_program() : void
    + step() : void
    + reset() : void
}

class CPU {
    - pc : PC
    - ir : IR
    - memory : Memory&
    - registers : Registers&
    - decoder : Decoder
    - decoded_instruction : Instruction
    - halted : bool
    + CPU(memory : Memory&, registers : Registers&)
    + validate_instruction() : bool
    + run_instruction() : void
    + is_halted() : bool
    + set_pc(address : uint8_t) : void
    + reset() : void
}

class Memory {
```

```
    - cells : vector<uint8_t>
    - screen : Screen&
    + Memory(screen : Screen&)
    + write(val : uint8_t, address : uint8_t) : void
    + read(address : uint8_t) : uint8_t
    + reset() : void
}

class Registers {
    - vals : vector<uint8_t>
    + read(address : uint8_t) : uint8_t
    + set(val : uint8_t, address : uint8_t) : void
    + reset() : void
}

class PC {
    - value : uint8_t
    + get() : uint8_t
    + set(value : uint8_t) : void
    + increment(amount : uint8_t) : void
}

class IR {
```

```
    - value : uint16_t
    + get() : uint16_t
    + set(value : uint16_t) : void
}

class Decoder {
    + decode(instruction : uint16_t) : Instruction
}

class Instruction {
    - opcode : uint8_t <<4-bit>>
    - r : uint8_t <<4-bit>>
    - x : uint8_t <<4-bit>>
    - y : uint8_t <<4-bit>>
    + Instruction(opcode : uint8_t = 0, r : uint8_t = 0, x : uint8_t =
0, y : uint8_t = 0)
    + get_opcode() : uint8_t
    + get_r() : uint8_t
    + get_x() : uint8_t
    + get_y() : uint8_t
}

class ProgramLoader {
```

```
    + load_program(file : string) : vector<uint16_t>
    + validate_program(program : vector<uint16_t>) : bool
}

class Screen {
    - output : string
    + write(val : uint8_t) : void
    + display() : void
    + reset() : void
}

Simulator *-- Screen
Simulator *-- Machine

Machine *-- Memory
Machine *-- Registers
Machine *-- CPU
Machine *-- ProgramLoader

CPU *-- PC
CPU *-- IR
CPU *-- Decoder
CPU --> Instruction : decoded_instruction
```

```
CPU --> Memory : memory reference

CPU --> Registers : registers reference


Decoder ..> Instruction : creates

Memory --> Screen : address FF writes output

Machine ..> Screen : constructor parameter


note right of Memory

  256 cells, addresses 00-FF.

  Address FF is memory-mapped

  to the screen.

end note


note right of Registers

  16 registers, R0-RF.

  Each register stores 8 bits.

end note


note right of Instruction

  Each field stores a value

  from 0 to 15.

  Instruction width: 16 bits.

end note
```

```
note right of CPU
  run_instruction(): fetch 2 bytes at PC,
  PC += 2, decode, validate, execute.
  Opcode C000 sets halted = true.
end note

note bottom of Machine
  run_program(): execute until halted.
  step(): execute exactly one instruction.
  load_program(): validate, split each 16-bit
  instruction into two bytes, set PC to start.
end note

note bottom of ProgramLoader
  Strip ; comments and empty lines.
  Require exactly four hex digits per instruction.
end note

@enduml
```

Be conservative. I prefer a simple, slightly less sophisticated design that I can fully understand and implement over a complex, theoretically elegant design

Appendix B: Gemini — Complete Conversation

Minimal Changes

Necessary Fixes

Machine & Memory Constructors: Updated to accept Screen& so that Simulator can pass its owned Screen instance down to Memory.

Machine::step(): Added a step method alongside run_program() to support single-instruction execution.

CPU::validate_instruction(): Added a validation method to CPU to ensure instructions are validated before execution.

Machine::run_program() Signature: Removed the redundant counter parameter since the program counter (PC) inside the CPU already manages execution state.

Optional Improvements (Excluded from Main Design)

Instruction History / Logging: Storing a log of executed instructions in the CPU or Simulator for debugging was left out to keep the design simple for a first-year student.

Updated UML Class Diagram

Code snippet

@startuml

```
title Vole Machine Simulator - UML Class Diagram

skinparam classAttributeIconSize 0

skinparam linetype ortho


class Simulator {

    - machine : Machine

    - screen : Screen

    + run() : void

    + show_main_menu() : void

    + reset() : void

}


class Machine {

    - cpu : CPU

    - memory : Memory

    - registers : Registers

    - loader : ProgramLoader

    + Machine(screen : Screen&)

    + load_program(start : uint8_t, file : string) : void

    + run_program() : void

    + step() : void

}
```

```
class CPU {
    - pc : PC
    - ir : IR
    - memory : Memory&
    - registers : Registers&
    - decoder : Decoder
    - decoded_instruction : Instruction
    + validate_instruction() : bool
    + run_instruction() : void
}

class Memory {
    - cells : vector<uint8_t>
    - screen : Screen&
    + Memory(screen : Screen&)
    + write(val : uint8_t, address : uint8_t) : void
    + read(address : uint8_t) : uint8_t
    + reset() : void
}

class Registers {
    - vals : vector<uint8_t>
    + read(address : uint8_t) : uint8_t
```

```
    + set(val : uint8_t, address : uint8_t) : void
    + reset() : void
}

class PC {
    - value : uint8_t
    + get() : uint8_t
    + set(value : uint8_t) : void
    + increment(amount : uint8_t) : void
}

class IR {
    - value : uint16_t
    + get() : uint16_t
    + set(value : uint16_t) : void
}

class Decoder {
    + decode(instruction : uint16_t) : Instruction
}

class Instruction {
    - opcode : uint8_t <<4-bit>>
```

```
    - r : uint8_t <<4-bit>>
    - x : uint8_t <<4-bit>>
    - y : uint8_t <<4-bit>>
    + get_opcode() : uint8_t
    + get_r() : uint8_t
    + get_x() : uint8_t
    + get_y() : uint8_t
}

class ProgramLoader {
    + load_program(file : string) : vector<uint16_t>
    + validate_program(program : vector<uint16_t>) : bool
}

class Screen {
    - output : string
    + write(val : uint8_t) : void
    + display() : void
    + reset() : void
}

Simulator *-- Machine
Simulator *-- Screen
```

```
Machine *-- CPU

Machine *-- Memory

Machine *-- Registers

Machine *-- ProgramLoader


CPU *-- PC

CPU *-- IR

CPU *-- Decoder

CPU --> "0..1" Instruction : decoded_instruction

CPU --> Memory : memory reference

CPU --> Registers : registers reference


Decoder ..> Instruction : creates


Memory --> Screen : address FF writes output


note right of Memory

    256 cells, addresses 00-FF.

    Address FF is memory-mapped

    to the screen.

end note
```

```
note right of Registers
    16 registers, R0-RF.
    Each register stores 8 bits.
end note

note right of Instruction
    Each field stores a value
    from 0 to 15.
    Instruction width: 16 bits.
end note

@enduml
```

Change Summary

What Changed:

Added Machine(screen : Screen&) and Memory(screen : Screen&) constructors to properly wire the memory-mapped screen reference.

Added step() to Machine to enable single-instruction execution mode.

Added validate_instruction() to CPU to formalize the validation requirement.

Cleaned up Machine::run_program() by removing the unnecessary counter parameter.

What Remained Unchanged:

All 11 original classes (Simulator, Machine, CPU, Memory, Registers, PC, IR, Decoder, Instruction, ProgramLoader, Screen).

Composition and ownership rules (Simulator owns Machine and Screen; Machine owns CPU, Memory, Registers, and ProgramLoader).

Core data structures (vector<uint8_t> for memory and registers, 16-bit instructions, 4-bit instruction fields).

Appendix C: ChatGPT — Complete Conversation

Vole Machine Simulator — UML Design Review
1. Brief assessment
The existing design is already reasonably organized for a first-year C++ OOP project. It does not need a new architecture, additional classes, inheritance, or design patterns.
The main issues are missing behavioral constraints and one UML relationship that does not accurately show ownership.
Genuine issues
1. Memory-mapped address FF needs explicit boundary rules.
Memory has 256 byte addresses, but FF is reserved for screen output.
The design must ensure that:
Loading a program never writes an instruction byte to FF.
Fetching a 16-bit instruction never uses FF as one of its two bytes.
The loader rejects programs that do not fit at the specified starting address.

The CPU must also detect an invalid fetch address instead of accidentally continuing through the memory-mapped screen or wrapping the PC around.
2. Program-loading failure behavior is underspecified.
Machine::load_program() returns bool, whereas ProgramLoader::load_program() returns a vector. The design should define how an unreadable file, malformed instruction, empty file, or oversized program is rejected, without partially loading an invalid program.
3. Halt-state handling needs a precise contract.
The halted attribute is appropriate, and the existing CPU::reset() method can clear it. However, successfully loading a new program must also clear the previous CPU halt state; otherwise, loading a program after executing C000 could leave the CPU halted.
The design should specify that:
C000 sets halted to true.
run_program() executes until the CPU halts.
step() executes at most one instruction and does not execute another instruction once halted.
Reset clears the halt state.
4. The decoded_instruction relationship is drawn incorrectly.
decoded_instruction is an Instruction object stored directly inside CPU, rather than an independently owned object. Its UML relationship should therefore be shown as composition, consistent with the other value members of CPU.
5. The division between program validation and instruction validation needs clarification.
ProgramLoader::validate_program() should handle program-level properties such as whether the program is empty or exceeds the overall instruction limit. CPU::validate_instruction() should handle supported opcodes and opcode-specific operand restrictions.
The decoder should only extract the four instruction fields; it should not decide whether an instruction is valid or execute it.
2. Minimal changes
Change
Classification
Why it is necessary
Add precise memory-boundary and loading constraints to the notes.
Necessary

Prevents program bytes from overwriting or overlapping the screen address.
Specify program-loader failure behavior.
Necessary
Makes the existing bool loading result meaningful and prevents partial loading.
Specify halt, step, and reset behavior.
Necessary
Ensures that execution state remains correct between programs.
Change the CPU–Instruction relationship to composition.
Necessary UML correction
Accurately represents the decoded_instruction value member.
Clarify the responsibility of each validation method.
Necessary clarification
Prevents duplicate validation and preserves the decoder's limited responsibility.
Make InstructionDecoder the consistent class name.
Naming clarification
The requirements refer to InstructionDecoder, while the supplied diagram calls it Decoder. The revised diagram uses InstructionDecoder; use the same name consistently throughout the project.
No additional classes, attributes, or methods are needed.
Optional improvements — excluded from the main design
Use std::array instead of std::vector. This can enforce the fixed memory and register counts at the type level. Keeping vectors is also acceptable if their sizes are initialized and maintained correctly.
Add a maximum instruction count to run_program(). This can prevent an infinite loop from running indefinitely, but it is not essential to the requested interface.
Add const qualifiers to read-only methods. This improves C++ correctness but does not require an architectural change.
Add a separate error-reporting class or execution-status abstraction. This is unnecessary for the current project; simple Boolean results for loading and ordinary C++ error handling are sufficient.
One instruction-set caution: common Vole specifications define opcode 6 as floating-point addition, with restricted operand formats for instructions such as 40RS, AR0X, and C000. The CPU should validate against your course's actual instruction table rather than assuming every four-nibble combination is legal.

```
3. Complete revised UML class diagram
@startuml
title Vole Machine Simulator - Revised UML Class Diagram

skinparam classAttributeIconSize 0
skinparam linetype ortho

class Simulator {
    - screen : Screen
    - machine : Machine
    + Simulator()
    + run() : void
    + show_main_menu() : void
    + reset() : void
}

class Machine {
    - memory : Memory
    - registers : Registers
    - cpu : CPU
    - loader : ProgramLoader
    + Machine(screen : Screen&)
    + load_program(start : uint8_t, file : string) : bool
    + run_program() : void
    + step() : void
    + reset() : void
}

class CPU {
    - pc : PC
    - ir : IR
    - memory : Memory&
    - registers : Registers&
    - decoder : InstructionDecoder
    - decoded_instruction : Instruction
    - halted : bool
    + CPU(memory : Memory&, registers : Registers&)
    + validate_instruction() : bool
    + run_instruction() : void
```

```
    + is_halted() : bool
    + set_pc(address : uint8_t) : void
    + reset() : void
}

class Memory {
    - cells : vector<uint8_t>
    - screen : Screen&
    + Memory(screen : Screen&)
    + write(val : uint8_t, address : uint8_t) : void
    + read(address : uint8_t) : uint8_t
    + reset() : void
}

class Registers {
    - vals : vector<uint8_t>
    + read(address : uint8_t) : uint8_t
    + set(val : uint8_t, address : uint8_t) : void
    + reset() : void
}

class PC {
    - value : uint8_t
    + get() : uint8_t
    + set(value : uint8_t) : void
    + increment(amount : uint8_t) : void
}

class IR {
    - value : uint16_t
    + get() : uint16_t
    + set(value : uint16_t) : void
}

class InstructionDecoder {
    + decode(instruction : uint16_t) : Instruction
}

class Instruction {
```

```
    - opcode : uint8_t <<4-bit>>
    - r : uint8_t <<4-bit>>
    - x : uint8_t <<4-bit>>
    - y : uint8_t <<4-bit>>
    + Instruction(opcode : uint8_t = 0, r : uint8_t = 0, x : uint8_t =
0, y : uint8_t = 0)
    + get_opcode() : uint8_t
    + get_r() : uint8_t
    + get_x() : uint8_t
    + get_y() : uint8_t
}

class ProgramLoader {
    + load_program(file : string) : vector<uint16_t>
    + validate_program(program : vector<uint16_t>) : bool
}

class Screen {
    - output : string
    + write(val : uint8_t) : void
    + display() : void
    + reset() : void
}

' Simulator owns both objects.
Simulator *-- Screen
Simulator *-- Machine

' Machine owns its main components.
Machine *-- Memory
Machine *-- Registers
Machine *-- CPU
Machine *-- ProgramLoader

' CPU owns these value members.
CPU *-- PC
CPU *-- IR
CPU *-- InstructionDecoder
CPU *-- Instruction : decoded_instruction
```

```
' These are non-owning references to Machine-owned objects.
CPU --> "1" Memory : memory reference
CPU --> "1" Registers : registers reference

' The decoder creates and returns an Instruction value.
InstructionDecoder ..> Instruction : creates / returns

' Memory uses the same Screen object supplied by Simulator.
Memory --> "1" Screen : screen reference

' Machine receives Screen through its constructor.
Machine ..> Screen : constructor parameter

note right of Memory
  Exactly 256 byte cells,
  addresses 00-FF.

  FF is reserved for screen output.
  A write to FF calls Screen::write()
  instead of storing the value in cells.

  Loading and instruction fetching must
  never use FF as an instruction byte.
end note

note right of Registers
  Exactly 16 registers, R0-RF.
  Each register stores 8 bits.

  read() and set() use register
  addresses from 0 to 15.
end note

note right of Instruction
  Four fields, each representing
  a value from 0 to 15.

  Each field is stored using uint8_t.
```

```
  Instruction width: 16 bits.
end note

note right of InstructionDecoder
  Extract the four 4-bit fields
  from the 16-bit instruction.

  Do not validate or execute
  instruction semantics here.
end note

note right of CPU
  run_instruction():

  1. Fetch two bytes at PC.
  2. Advance PC by two bytes.
  3. Decode the 16-bit instruction.
  4. Validate the decoded instruction.
  5. Execute it only if valid.

  C000 sets halted = true.
  Further execution is blocked
  while halted remains true.

  Reject a fetch whose two bytes would
  overlap the screen-mapped address FF.
end note

note bottom of Machine
  load_program():

  1. Load and validate the complete file.
  2. Check that the program fits at start.
  3. Do not write any instruction byte to FF.
  4. Load the instructions as two bytes,
     high byte first.
  5. Reset CPU execution state and set PC
     to start after a successful load.
```

```
  Return true on success and false on failure.
  A failed load must not partially load the
  new program.

  run_program(): execute until halted.
  step(): execute at most one instruction;
  do nothing if the CPU is already halted.

  reset(): reset CPU, Memory and Registers.
end note

note bottom of ProgramLoader
  Ignore empty lines and whitespace.

  Remove comments beginning with ;
  through the end of their line.

  Each remaining non-empty line must
  contain exactly four hexadecimal digits.

  Return a complete parsed program.
  An unreadable, malformed or empty program
  must be reported as a loading failure.

  validate_program() checks program-level
  constraints, such as non-empty input
  and the maximum overall program size.
  It does not validate instruction semantics.
end note

note bottom of Screen
  write(val): append one output byte.
  display(): display accumulated output.
  reset(): clear accumulated output.
end note

note bottom of Simulator
  screen must be constructed before machine
  and remain alive for the lifetime of machine.
```

```
  reset() resets the machine and screen.
end note

@enduml
```

4. Change summary

Changed

Standardized the decoder class name as InstructionDecoder; its responsibility remains unchanged.

Changed the CPU–Instruction link to composition because decoded_instruction is stored by value.

Specified memory-boundary constraints for address FF.

Specified how invalid files, empty programs, and programs that do not fit are rejected.

Clarified the fetch-decode-validate-execute sequence and halt behavior.

Specified that a successful program load clears the previous CPU execution state before setting the PC to the requested start address.

Clarified that program-level validation belongs to ProgramLoader, while instruction validation belongs to CPU.

Remained unchanged

All original classes and their core responsibilities.

Simulator ownership of Machine and Screen.

Machine ownership of Memory, Registers, CPU, and ProgramLoader.

CPU references to the same Memory and Registers objects owned by Machine.

Memory's reference to the same Screen object owned by Simulator.

The original public method signatures and the existing PC, IR, Instruction, Memory, Registers, Screen, and ProgramLoader responsibilities.

The use of ordinary C++ composition, value members, and references instead of additional abstraction layers.

5. Implementation difficulty

The revised design remains suitable for a first-year CS student. It uses familiar C++ concepts: classes, constructors, composition, references, vectors, fixed-size invariants, and simple methods. No inheritance, templates beyond ordinary containers, factories, or design patterns are introduced.

The most challenging part remains implementing the CPU's instruction execution correctly. That difficulty comes from the simulator's behavior itself, not from unnecessary complexity in the UML.
The recommended implementation order is Instruction and InstructionDecoder, then PC and IR, followed by Memory, Registers, Screen, and ProgramLoader. Implement and test the CPU next, and finish by connecting everything through Machine and Simulator.
The key outcome is that the design stays close to your original one while making the execution, loading, and memory-boundary rules explicit before implementation begins.
Resolve the FF address contradictionClarify composition and reference ownership

Appendix D: Claude — Complete Conversation

Your design is already close to correct. Ownership, the reference-based wiring, and the split between Decoder, CPU, ProgramLoader and Screen are all sound, so I left them alone.
1. Brief assessment (genuine problems only)
ProgramLoader can't do what its note says. validate_program(vector<uint16_t>) runs after parsing, so it can no longer check "exactly four hex digits". For example, "12" and "0012" both become the number 0x12. load_program returns only a vector, so it also can't report a missing file or a malformed line. And nothing checks that the program fits in memory, because the start address isn't available to the loader.
Registers has no constructor. Without one, vals is an empty vector, so the first register access is out of bounds. Memory already has a constructor that sizes it, so Registers needs the same.
PC.value, IR.value and CPU.halted have no initial value. A garbage halted could stop the machine before it starts.
CPU --> Instruction is the wrong relationship. CPU holds decoded_instruction by value, so this is composition, like PC, IR and Decoder.
Some behaviors are undefined. These are not structural flaws, just things the design never states:
what happens when validation fails;

what happens when you load a new program after a halt;
byte order of an instruction in memory;
whether address FF also stores its value;
what stops a program from being loaded over address FF.
2. Minimal changes
Necessary:
Change
Why
ProgramLoader.load_program(file, program : vector<uint16_t>&) : bool
Parses the text, where the four-digit check is still possible, and can report failure.
ProgramLoader.validate_program(program, start : uint8_t) : bool
Same method kept, now able to check that the program fits. The program must end before address FF, because loading a byte into FF would print to the screen.
Add Registers() constructor
Sizes the vector to 16 so register access is valid.
Defaults value = 0 for PC and IR, halted = false for CPU
Prevents uninitialized values.
CPU *-- Instruction replaces CPU --> Instruction
Matches the by-value member.
Updated notes
Define the rules listed in problem 5 (see the rules below).
Rules now stated in the notes (no new classes or methods):
Invalid instruction: halted = true and the instruction is not executed. This also stops a program that runs past its end into zeroed memory, because 0000 has opcode 0, which is invalid. If you'd rather skip bad instructions and continue, that's a one-line change in the note.
Halt instruction: C000 is valid only if the other three fields are zero.
Halted CPU: run_instruction() does nothing, so step() is safe after a halt.
Loading: Machine::load_program calls cpu.reset() and then cpu.set_pc(start), so a new program isn't blocked by an old halt.
Byte order: the high byte goes at the lower address.
Address FF: the value is stored in the cell and also sent to the screen.
Optional (left out of the diagram):

Machine.is_halted() so Simulator can say "program finished" after a step.
Read-only getters (PC, registers, memory) if the menu should show machine state after each step.
Private fetch() and execute() helpers inside CPU to make the cycle more visible. The note already describes it, so this is purely style.
Private helper methods for opcode 6 (floating-point add), if you support it. That's an implementation detail, not a design change.
Renaming Decoder to InstructionDecoder. You referred to it by that name, but I kept the diagram's name.
3. Updated UML

```
@startuml
title Vole Machine Simulator - Revised UML Class Diagram
skinparam classAttributeIconSize 0
skinparam linetype ortho

class Simulator {
    - screen : Screen
    - machine : Machine
    + Simulator()
    + run() : void
    + show_main_menu() : void
    + reset() : void
}

class Machine {
    - memory : Memory
    - registers : Registers
    - cpu : CPU
    - loader : ProgramLoader
    + Machine(screen : Screen&)
    + load_program(start : uint8_t, file : string) : bool
    + run_program() : void
    + step() : void
    + reset() : void
}

class CPU {
    - pc : PC
```

```
    - ir : IR
    - memory : Memory&
    - registers : Registers&
    - decoder : Decoder
    - decoded_instruction : Instruction
    - halted : bool = false
    + CPU(memory : Memory&, registers : Registers&)
    + validate_instruction() : bool
    + run_instruction() : void
    + is_halted() : bool
    + set_pc(address : uint8_t) : void
    + reset() : void
}

class Memory {
    - cells : vector<uint8_t>
    - screen : Screen&
    + Memory(screen : Screen&)
    + write(val : uint8_t, address : uint8_t) : void
    + read(address : uint8_t) : uint8_t
    + reset() : void
}

class Registers {
    - vals : vector<uint8_t>
    + Registers()
    + read(address : uint8_t) : uint8_t
    + set(val : uint8_t, address : uint8_t) : void
    + reset() : void
}

class PC {
    - value : uint8_t = 0
    + get() : uint8_t
    + set(value : uint8_t) : void
    + increment(amount : uint8_t) : void
}

class IR {
```

```
    - value : uint16_t = 0
    + get() : uint16_t
    + set(value : uint16_t) : void
}

class Decoder {
    + decode(instruction : uint16_t) : Instruction
}

class Instruction {
    - opcode : uint8_t <<4-bit>>
    - r : uint8_t <<4-bit>>
    - x : uint8_t <<4-bit>>
    - y : uint8_t <<4-bit>>
    + Instruction(opcode : uint8_t = 0, r : uint8_t = 0, x : uint8_t =
0, y : uint8_t = 0)
    + get_opcode() : uint8_t
    + get_r() : uint8_t
    + get_x() : uint8_t
    + get_y() : uint8_t
}

class ProgramLoader {
    + load_program(file : string, program : vector<uint16_t>&) : bool
    + validate_program(program : vector<uint16_t>, start : uint8_t) :
bool
}

class Screen {
    - output : string
    + write(val : uint8_t) : void
    + display() : void
    + reset() : void
}

Simulator *-- Screen
Simulator *-- Machine

Machine *-- Memory
```

```
Machine *-- Registers
Machine *-- CPU
Machine *-- ProgramLoader

CPU *-- PC
CPU *-- IR
CPU *-- Decoder
CPU *-- Instruction : decoded_instruction
CPU --> Memory : memory reference
CPU --> Registers : registers reference

Decoder ..> Instruction : creates
Memory --> Screen : address FF writes output
Machine ..> Screen : constructor parameter

note right of Memory
  256 cells, addresses 00-FF.
  write(): store val in the cell; if address
  is FF, also call screen.write(val).
  read(): return the stored cell.
  reset() clears cells only, not the screen.
end note

note right of Registers
  16 registers, R0-RF.
  Each register stores 8 bits.
  Constructor creates 16 registers set to 0.
end note

note right of Instruction
  Each field stores a value
  from 0 to 15.
  Instruction width: 16 bits.
end note

note right of CPU
  run_instruction(): does nothing if halted.
  Fetch: high byte at PC, low byte at PC+1
  (stored in IR). PC += 2. Decode, validate.
```

```
  If invalid: halted = true, do not execute.
  Otherwise execute.
  Opcode C (C000, other fields must be 0)
  sets halted = true.
  reset(): PC = 0, IR = 0, halted = false.
end note

note bottom of Machine
  run_program(): while not halted, run_instruction().
  step(): one run_instruction().
  load_program(): loader.load_program, then
  loader.validate_program. If either fails,
  return false and leave memory unchanged.
  Otherwise write each instruction as two bytes
  (high byte at address, low byte at address+1)
  starting at start, then cpu.reset() and
  cpu.set_pc(start).
  reset(): memory, registers, cpu.
  Screen is reset by Simulator.reset().
end note

note bottom of ProgramLoader
  load_program(): strip text after ';', trim,
  skip empty lines. Each remaining line must be
  exactly four hex digits (one instruction per
  line). Return false on file or format error.
  validate_program(): program must fit in memory
  starting at start and end before address FF
  (FF is the screen).
end note

@enduml
4. Change summary
Changed:
ProgramLoader: signature of load_program (reference output, bool
result) and validate_program (added start).
Registers: added + Registers().
PC.value, IR.value and CPU.halted: added default values.
CPU and Instruction: relationship changed from --> to *--.
```

Notes for Memory, Registers, CPU, Machine and ProgramLoader.
Unchanged: all other classes, attributes, methods and relationships, including the Simulator/Machine/Screen ownership and the Memory-to-Screen reference.
5. Implementation difficulty
It's still a good fit for a beginner, since there are no new classes, no inheritance, and no new patterns. The only new C++ idea is passing the program vector by reference as an output parameter, which is a basic technique.
Two things to watch when you implement it:
Declaration order: inside Machine, declare memory and registers before cpu, since cpu is built from references to them (your diagram already has this order). Likewise in Simulator, declare screen before machine.
Address arithmetic: uint8_t values promote to int in expressions like pc + 1, so cast the result back when you pass it as an address.

5. Examples

This section presents the two Vole machine programs we wrote to evaluate the simulator. Both follow the program file format: comment lines start with `;`, empty lines are ignored, and each instruction is on its own line as four hexadecimal digits. Both programs are loaded at start address `00`, and each instruction occupies two memory cells. The screen is memory-mapped at address `FF`, so a store to `FF` writes to the screen.

5.1 Program 1: Conditional Jump Test

Purpose: This program tests the JUMP instruction (opcode `B`). It loads the value 5 into R0 and R1, jumps if R1 equals R0, and then displays the value of R1 on the screen. It exercises opcodes `2`, `B`, `3` (store to the screen address `FF`) and `C`.

Program file (program1):

```
; Vole Machine Program: Conditional Jump & Comparison Test

; Purpose: Load reference value 5 into R0, load 5 into R1,

; compare R1 with R0 (R0 holds 5), and jump if equal.


2005 ; Load immediate value 05 into register 0 (reference for
comparison)

2105 ; Load immediate value 05 into register 1

B106 ; Jump to instruction at address 06 if register 1 equals register
0

C000 ; Halt execution (if jump didn't happen)

3100 ; Store register 1 into memory [00] (screen output if jump
succeeded and reached here)

C000 ; Halt execution
```

Memory layout:

Address	Instruction	Meaning
00	2005	R0 = 05
02	2105	R1 = 05
04	B108	R1 equals R0, so PC =08
06	C000	Skipped

```
08      31FF      Screen shows R1 =
                  05

0A      C000      Halt
```

Expected result: The screen displays 05. Final state: R0 = 05, R1 = 05, PC = 0C.

Second test case: Changing 2105 to 2106 makes R1 differ from R0, so the jump is not taken. The program then executes the C000 at address 06 and halts with nothing displayed on the screen (final PC = 08). Running both versions tests both branches of the JUMP instruction.

5.2 Program 2: Addition and Output

Purpose: This program tests the arithmetic and output path. It loads 10 (0A) into R0 and 5 into R1, adds them as two's complement values, and displays the result on the screen. It exercises opcodes 2, 5, 3 and C.

Program file (program2):

```
 Vole Machine Program 2: Addition and Output

; Purpose: Load 10 (0A) into R0 and 5 into R1, add them,

; and display the result (15 = 0F) on the screen.


200A ; Load immediate value 0A (10) into register 0

2105 ; Load immediate value 05 into register 1

5301 ; Add registers 0 and 1, put result in register 3

33FF ; Store register 3 to address FF (screen output)

C000 ; Halt execution


Memory layout:
```

Address	Instruction	Meaning
00	200A	R0 = 0A
02	2105	R1 = 05
04	5301	R3 = 0A + 05 = 0F
06	33FF	Screen shows R3 = 0F
08	C000	Halt

Expected result: The screen displays 0F (15 in decimal). Final state: R0 = 0A, R1 = 05, R3 = 0F, PC = 0A.

5.3 How the Programs Are Used?

Together, the two programs cover loading immediate values, addition, conditional jump (taken and not taken), writing to the screen, and halting. We use them to check that the simulator loads a program at an even address, fetches and decodes each instruction, executes it correctly, and displays the memory, PC, IR, registers and screen after the run or after each step.

6.Video Link

https://drive.google.com/file/d/1_cps_n04zd3eCUvT-Su9-iiFoP_kMc-h/view?usp=drive_link

7.Breakdown (who did what in code)

- **Omar Wael Omar Abd Al-Latif**
  - **IR.h, IR.cpp, PC.h, PC.cpp, CPU.h, CPU.cpp, Instruction.h, Instruction.cpp, floating_point, floating_point.cpp and Utlities.cpp.**

- **Kirolos Nabil Ataa Allah**
  - **Registers.h, Registers.cpp, Machine.h, Machine.cpp.**

- **Seif Sabry Abd Al-Salam**
  - **ProgramLoader.h, ProgramLoader.cpp, Memory.h, Memory.cpp, Decoder.cpp and Decoder.h.**

- **Mohamed Ahmed Mohamed Mostafa**
  - **Simulator.h, Simulator.cpp, Screen.h and Screen.cpp**

8. Final Design After Code Implementation

8.1 Updated PlantUML Code (Post-Implementation)

```
@startuml
title Vole Machine Simulator - Final UML Class Diagram (as implemented
in Part 2)
skinparam classAttributeIconSize 0
skinparam linetype ortho

class Simulator {
- screen : Screen
- machine : Machine
+ Simulator()
+ run() : void
+ reset() : void
- show_main_menu() : void
- show_load_program_menu() : void
- show_run_program_menu() : void
- show_run_line_by_line_menu() : void
- perform_main_menu_choice(choice : short) : void
}

class Machine {
- memory : Memory
- registers : Registers
- cpu : CPU
- loader : ProgramLoader
+ is_program_loaded : bool = false
+ Machine(screen : Screen&)
+ load_program(start : uint8_t, program_path : const string&) : bool
+ run_program() : bool
+ step() : bool
+ is_halted() : bool
+ show_status() : void
```

```
+ reset() : void
}

class CPU {
- pc : PC
- ir : IR
- memory : Memory&
- registers : Registers&
- decoder : Decoder
- decoded_instruction : Instruction
- halted : bool = false
- validate_instruction() : bool
+ CPU(memory : Memory&, registers : Registers&)
+ run_instruction() : bool
+ is_halted() : bool
+ set_pc(address : uint8_t) : void
+ show() : void
+ reset() : void
}

class Memory {
- cells : vector<uint8_t>
- screen : Screen&
+ Memory(screen : Screen&)
+ write(val : uint8_t, address : uint8_t) : void
+ read(address : uint8_t) : uint8_t
+ show() : void
+ reset() : void
}

class Registers {
- vals : vector<uint8_t>
+ Registers()
+ read(address : uint8_t) : uint8_t
+ set(val : uint8_t, address : uint8_t) : void
+ show() : void
+ reset() : void
}
```

```
class PC {
- value : uint8_t = 0
+ get() : uint8_t
+ set(value : uint8_t) : void
+ increment(amount : uint8_t) : void
}

class IR {
- value : uint16_t = 0
+ get() : uint16_t
+ set(value : uint16_t) : void
}

class Decoder {
+ decode(instruction : uint16_t) : Instruction
}

class Instruction {
- opcode : uint8_t <<4-bit>>
- r : uint8_t <<4-bit>>
- x : uint8_t <<4-bit>>
- y : uint8_t <<4-bit>>
+ Instruction(opcode : uint8_t = 0, r : uint8_t = 0, x : uint8_t = 0,
y : uint8_t = 0)
+ get_opcode() : uint8_t
+ get_r() : uint8_t
+ get_x() : uint8_t
+ get_y() : uint8_t
}

class ProgramLoader {
- validate_program(program : const vector<uint16_t>&, start_address :
uint8_t) : bool
+ load_program(program_path : const string&, start_address : uint8_t,
memory : Memory&) : bool
}

class Screen {
- output : string
```

```
+ write(val : uint8_t) : void
+ display() : void
+ reset() : void
}

class floating_point {
- number : uint8_t
- floated_number : float
- to_float(_number : uint8_t) : float
+ floating_point(_number : uint8_t = 0)
+ to_bits() : uint8_t
+ operator+(other : const floating_point&) : floating_point
+ get_float() : float
+ set_float(value : float) : void
}

class Utilities <<utility, free functions>> {
+ show_error_message(message : const string&) : void
+ show_success_message(message : const string&) : void
+ is_whitespace_only(str : const string&) : bool
+ left_trim(str : string) : string
+ right_trim(str : string) : string
+ trim(str : string) : string
+ convert_file_to_vector(file_path : const string&) : vector<uint16_t>
+ read_number(prompt : const string&) : int
+ read_choice(prompt : const string&, error_message : const string&,
from : int, to : int) : int
+ read_file_path() : string
+ read_address() : uint8_t
}

Simulator *-- Screen
Simulator *-- Machine
Machine *-- Memory
Machine *-- Registers
Machine *-- CPU
Machine *-- ProgramLoader
CPU *-- PC
CPU *-- IR
```

```
CPU *-- Decoder
CPU *-- Instruction : decoded_instruction
CPU --> Memory : memory reference
CPU --> Registers : registers reference
Decoder ..> Instruction : creates
Memory --> Screen : address FF writes output
Machine ..> Screen : constructor parameter
ProgramLoader ..> Memory : writes program bytes
CPU ..> floating_point : opcode 6 (float add)
Simulator ..> Utilities : menus / input
Machine ..> Utilities : error messages
ProgramLoader ..> Utilities : convert_file_to_vector()

note right of Memory
256 cells, addresses 00-FF.
write(): stores val in the cell; if address
is FF it also calls screen.write(val).
read(): returns the stored cell.
reset() clears cells only, not the screen.
show(): prints 16x16 hex matrix.
end note

note right of Registers
16 registers, R0-RF, 8 bits each.
Constructor creates 16 registers set to 0.
end note

note right of CPU
run_instruction():
1. Fetch high byte at PC, low byte at PC+1 into IR.
2. Decode.
3. validate_instruction(): throws runtime_error
   for an invalid instruction (PC not advanced,
   halted stays false, Machine returns false).
4. PC += 2, execute.
Opcode C000 sets halted = true.
reset(): PC = 0, IR = 0, halted = false.
end note
```

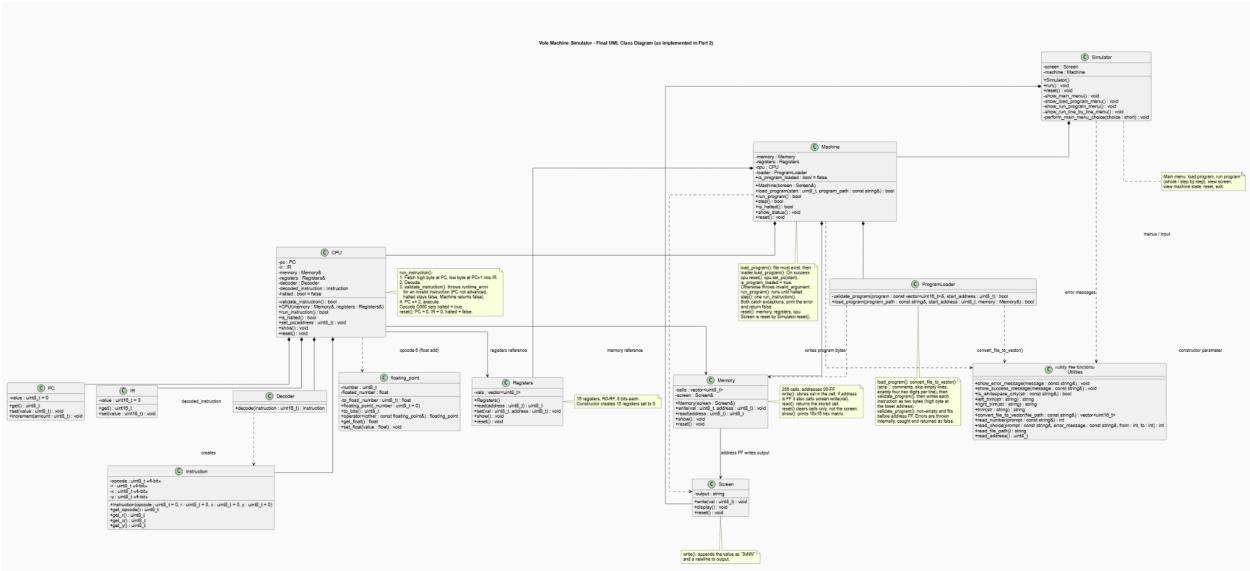
```
note bottom of Machine
load_program(): file must exist, then
loader.load_program(). On success:
cpu.reset(), cpu.set_pc(start),
is_program_loaded = true.
Otherwise throws invalid_argument.
run_program(): runs until halted.
step(): one run_instruction().
Both catch exceptions, print the error
and return false.
reset(): memory, registers, cpu.
Screen is reset by Simulator.reset().
end note

note bottom of ProgramLoader
load_program(): convert_file_to_vector()
(strip ';' comments, skip empty lines,
exactly four hex digits per line), then
validate_program(), then writes each
instruction as two bytes (high byte at
the lower address).
validate_program(): non-empty and fits
before address FF. Errors are thrown
internally, caught and returned as false.
end note

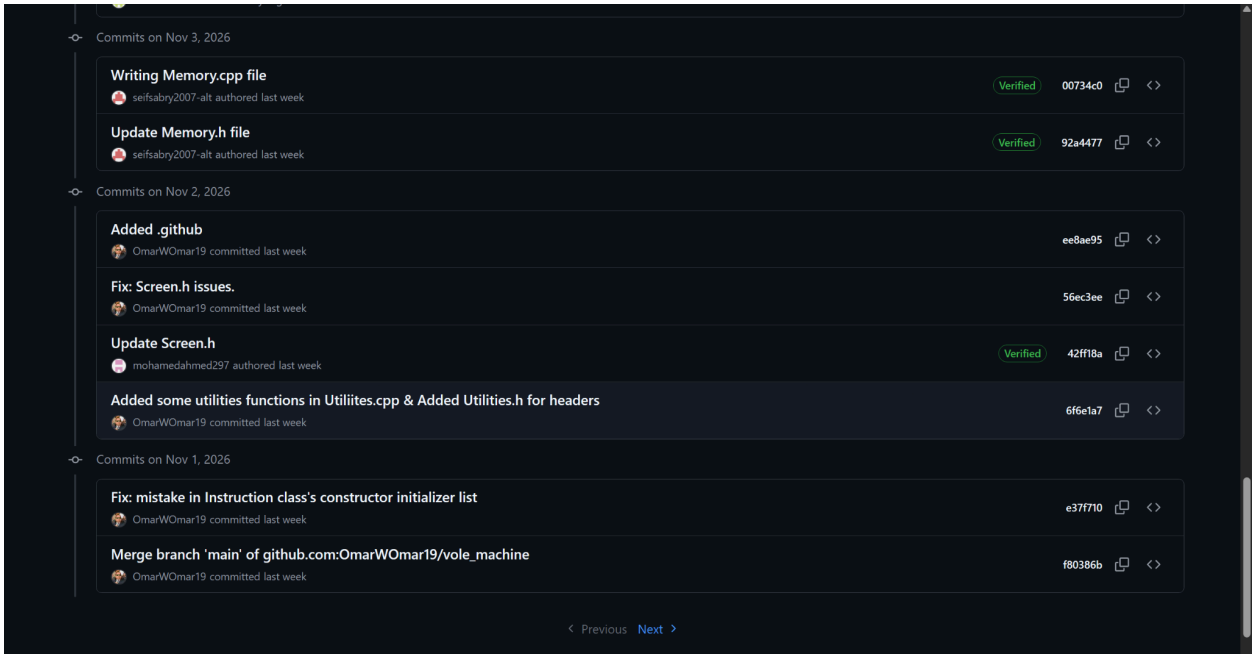
note bottom of Screen
write(): appends the value as "0xNN"
and a newline to output.
end note

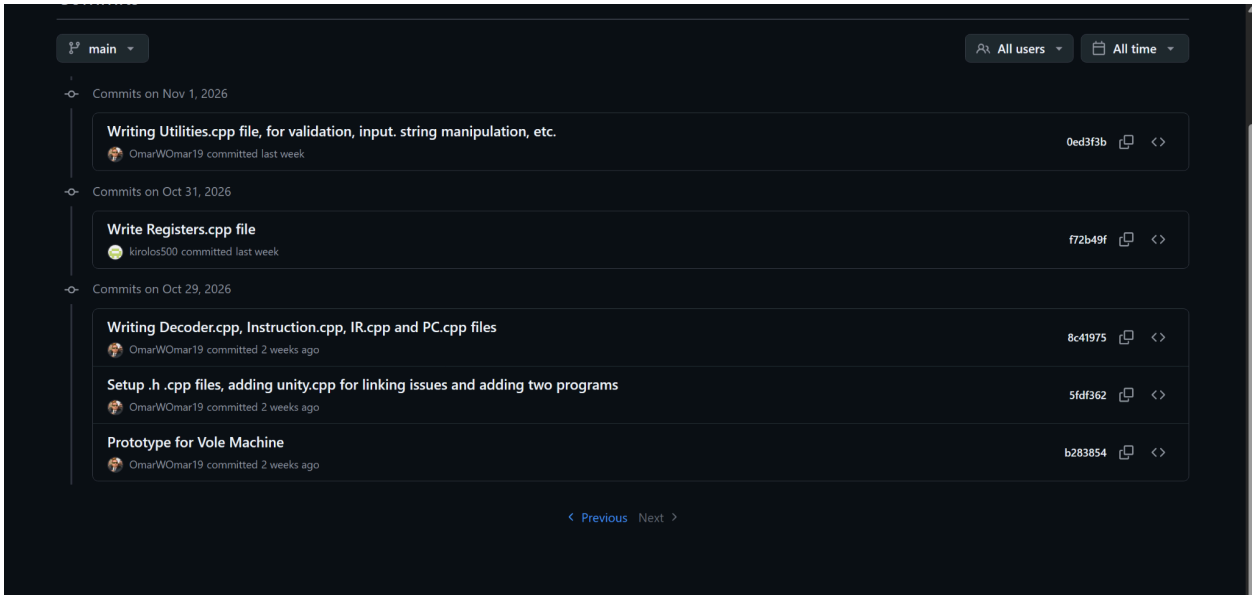
note bottom of Simulator
Main menu: load program, run program
(whole / step by step), view screen,
view machine state, reset, exit.
end note
@enduml
```

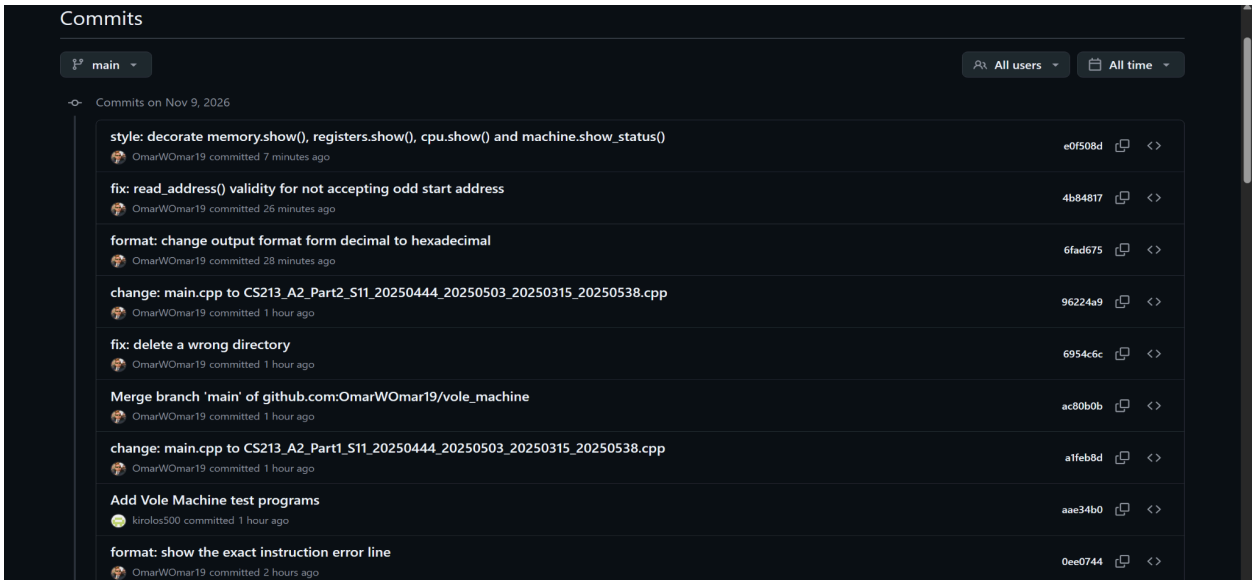
8.2 Final Post-Implementation UML Class Diagram

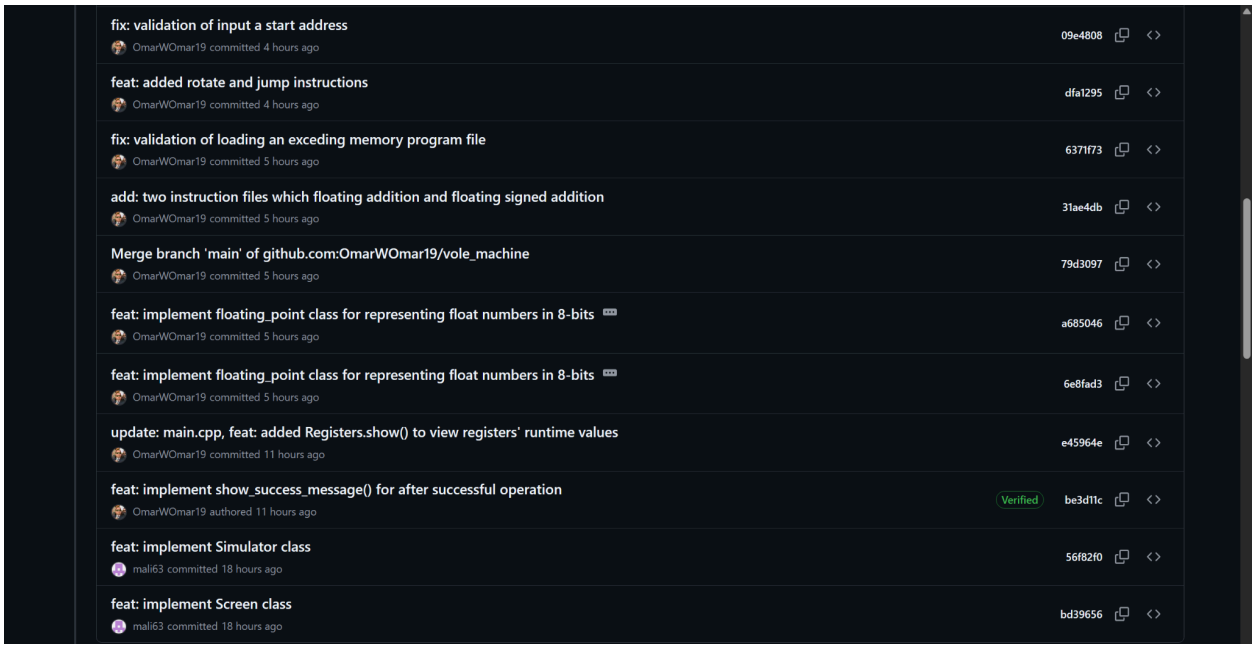


9. GitHub Screenshots


Commits on Nov 3, 2026
Writing Memory.cpp file
seifsabry2007-alt authored last week
Verified
00734c0
Update Memory.h file
seifsabry2007-alt authored last week
Verified
92a4477
Commits on Nov 2, 2026
Added .github
OmarWOmar19 committed last week
ee8ae95
Fix: Screen.h issues.
OmarWOmar19 committed last week
56ec3ee
Update Screen.h
mohamedahmed297 authored last week
Verified
42ff18a
Added some utilities functions in Utiliites.cpp & Added Utilities.h for headers
OmarWOmar19 committed last week
6f6e1a7
Commits on Nov 1, 2026
Fix: mistake in Instruction class's constructor initializer list
OmarWOmar19 committed last week
e37f710
Merge branch 'main' of github.com:OmarWOmar19/vole_machine
OmarWOmar19 committed last week
f80386b
Previous
Next

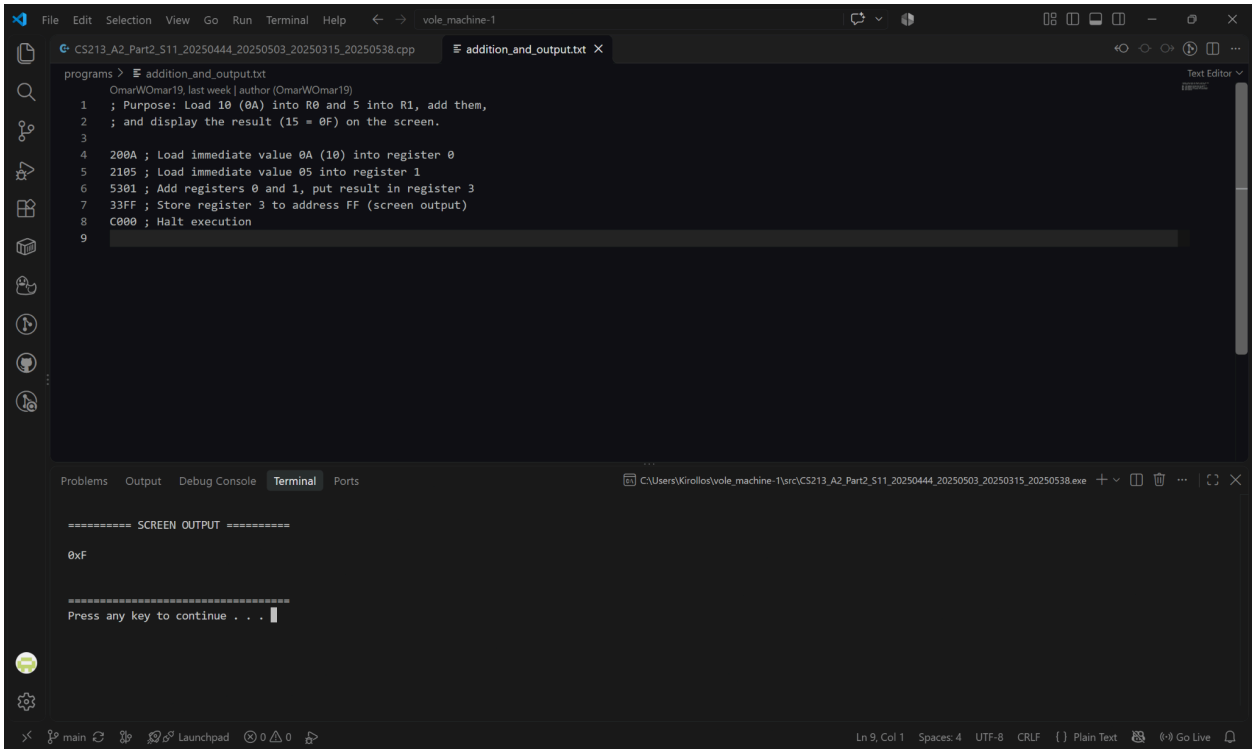

main
All users
All time
Commits on Nov 1, 2026
Writing Utilities.cpp file, for validation, input. string manipulation, etc.
OmarWOmar19 committed last week
0ed3f3b
Commits on Oct 31, 2026
Write Registers.cpp file
kirolos500 committed last week
f72b49f
Commits on Oct 29, 2026
Writing Decoder.cpp, Instruction.cpp, IR.cpp and PC.cpp files
OmarWOmar19 committed 2 weeks ago
8c41975
Setup .h .cpp files, adding unity.cpp for linking issues and adding two programs
OmarWOmar19 committed 2 weeks ago
5fdf362
Prototype for Vole Machine
OmarWOmar19 committed 2 weeks ago
b283854
Previous
Next


Commits
main
All users
All time
Commits on Nov 9, 2026
style: decorate memory.show(), registers.show(), cpu.show() and machine.show_status()
OmarWOmar19 committed 7 minutes ago
e0f508d
fix: read_address() validity for not accepting odd start address
OmarWOmar19 committed 26 minutes ago
4b84817
format: change output format form decimal to hexadecimal
OmarWOmar19 committed 28 minutes ago
6fad675
change: main.cpp to CS213_A2_Part2_S11_20250444_20250503_20250315_20250538.cpp
OmarWOmar19 committed 1 hour ago
96224a9
fix: delete a wrong directory
OmarWOmar19 committed 1 hour ago
6954c6c
Merge branch 'main' of github.com:OmarWOmar19/vole_machine
OmarWOmar19 committed 1 hour ago
ac80b0b
change: main.cpp to CS213_A2_Part1_S11_20250444_20250503_20250315_20250538.cpp
OmarWOmar19 committed 1 hour ago
a1feb8d
Add Vole Machine test programs
kirolos500 committed 1 hour ago
aae34b0
format: show the exact instruction error line
OmarWOmar19 committed 2 hours ago
0ee0744

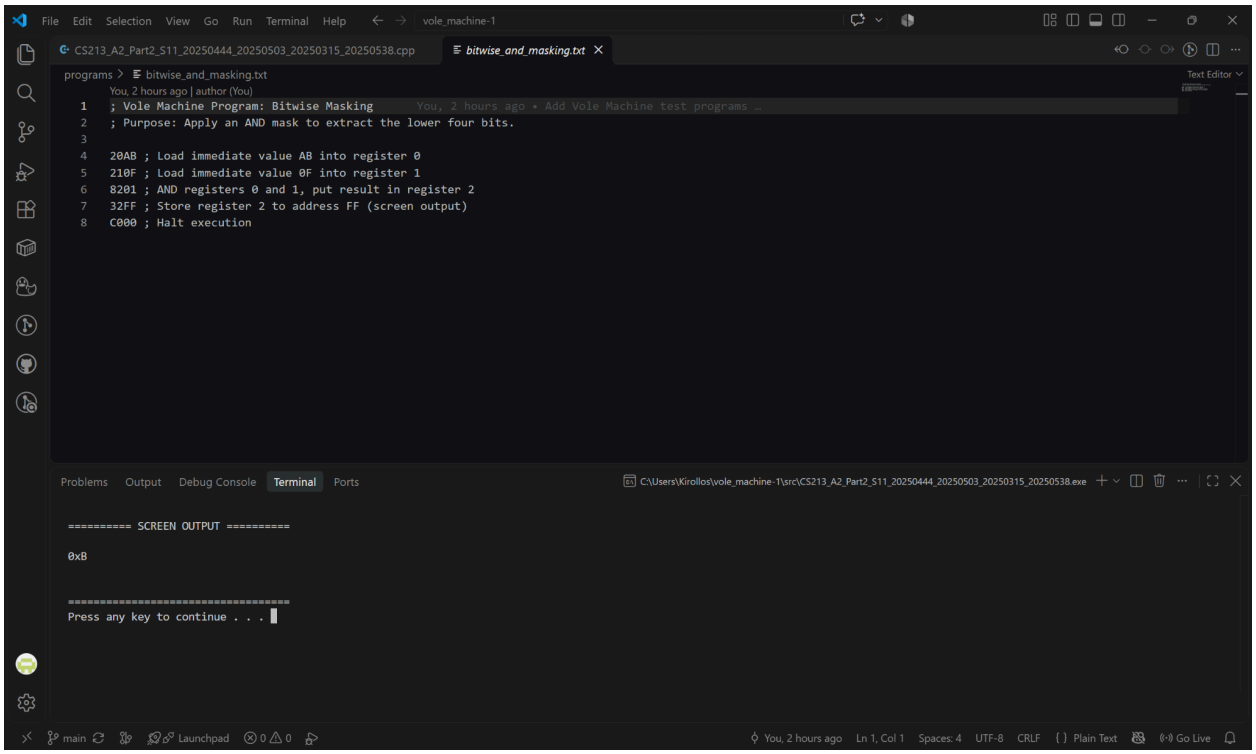

fix: validation of input a start address
OmarWOmar19 committed 4 hours ago
09e4808
feat: added rotate and jump instructions
OmarWOmar19 committed 4 hours ago
dfa1295
fix: validation of loading an exceding memory program file
OmarWOmar19 committed 5 hours ago
6371f73
add: two instruction files which floating addition and floating signed addition
OmarWOmar19 committed 5 hours ago
31ae4db
Merge branch 'main' of github.com:OmarWOmar19/vole_machine
OmarWOmar19 committed 5 hours ago
79d3097
feat: implement floating_point class for representing float numbers in 8-bits
OmarWOmar19 committed 5 hours ago
a685046
feat: implement floating_point class for representing float numbers in 8-bits
OmarWOmar19 committed 5 hours ago
6e8fad3
update: main.cpp, feat: added Registers.show() to view registers' runtime values
OmarWOmar19 committed 11 hours ago
e45964e
feat: implement show_success_message() for after successful operation
OmarWOmar19 authored 11 hours ago
Verified
be3d11c
feat: implement Simulator class
mali63 committed 18 hours ago
56f82f0
feat: implement Screen class
mali63 committed 18 hours ago
bd39656

10. Test Screenshots

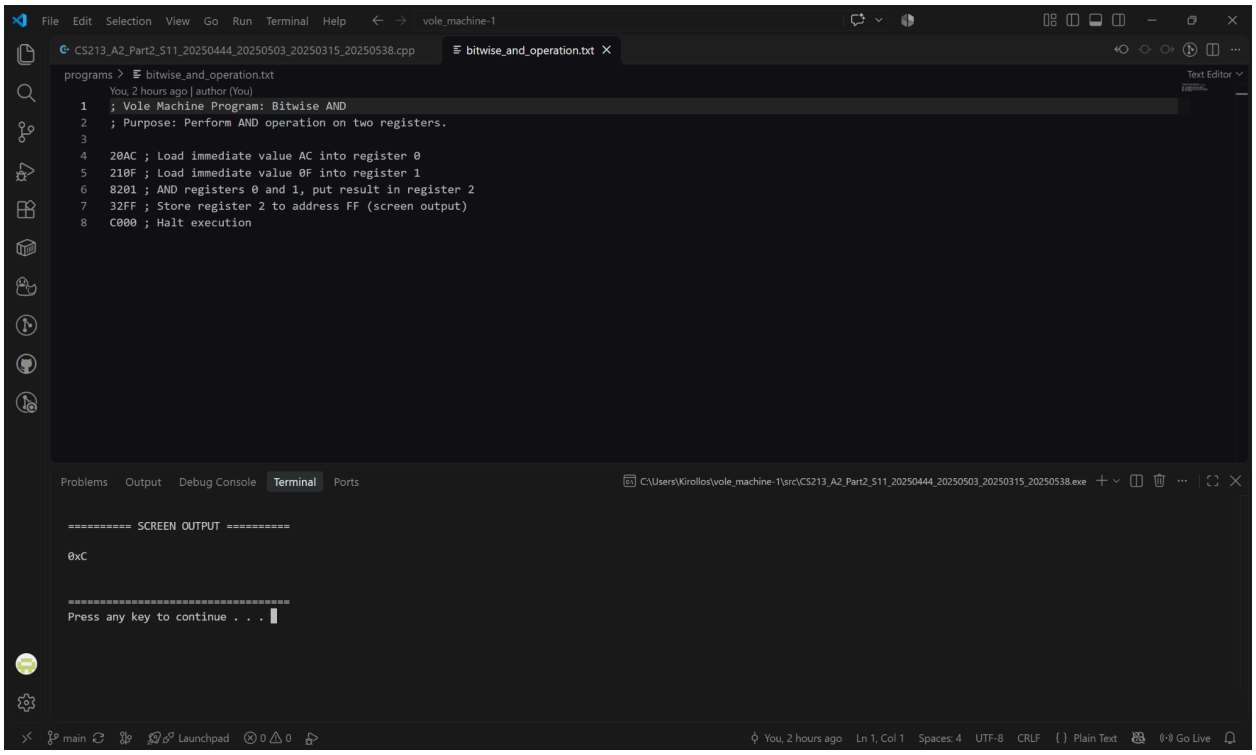
Screen 1



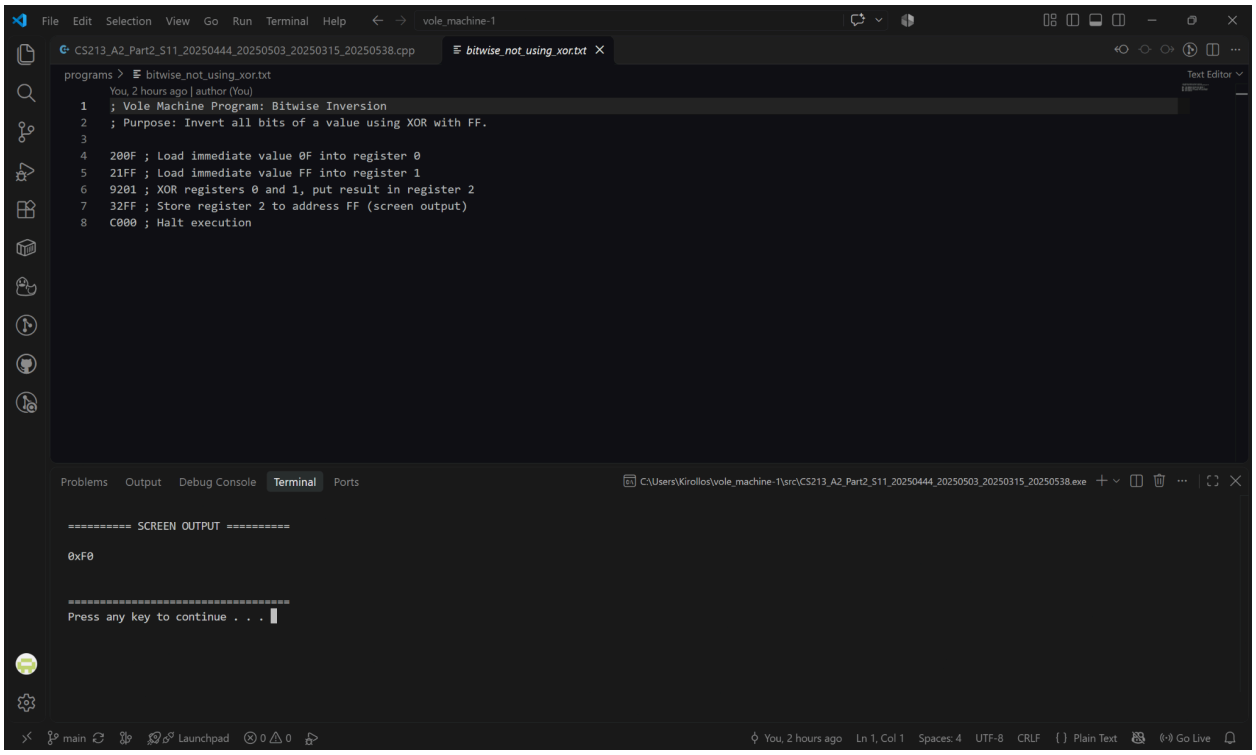
Screen 2



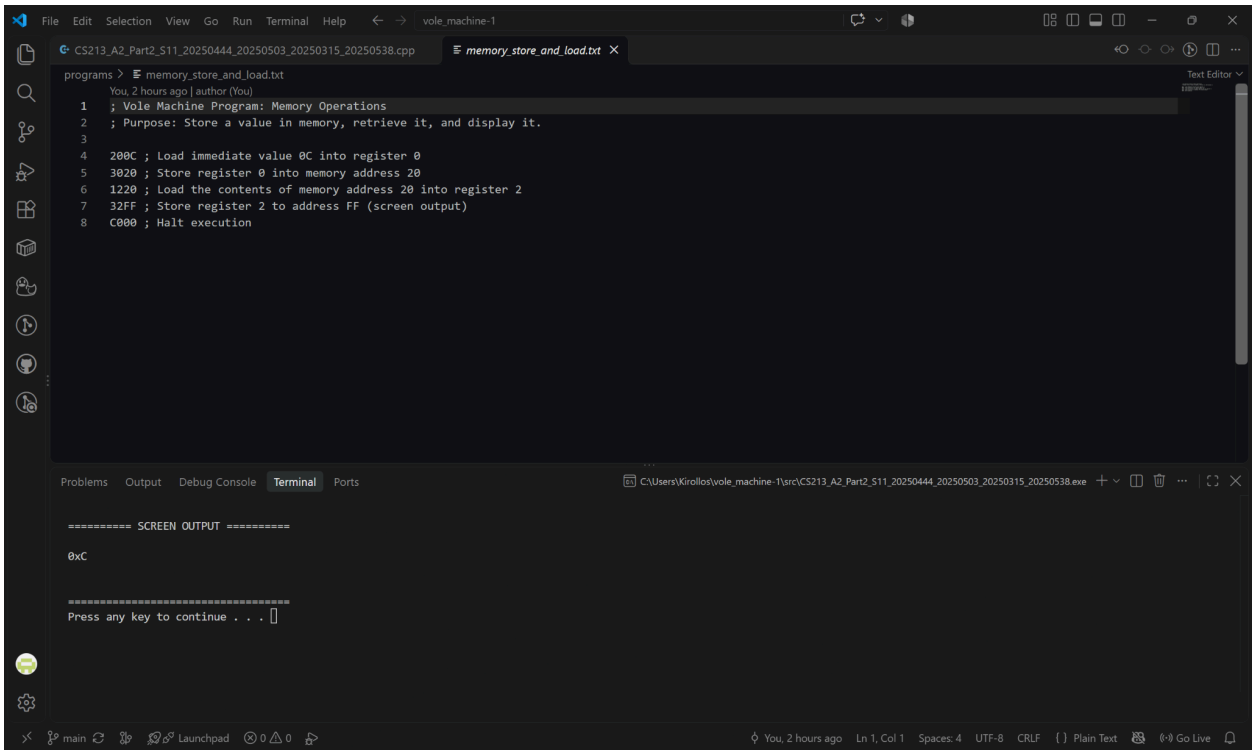
Screen 3



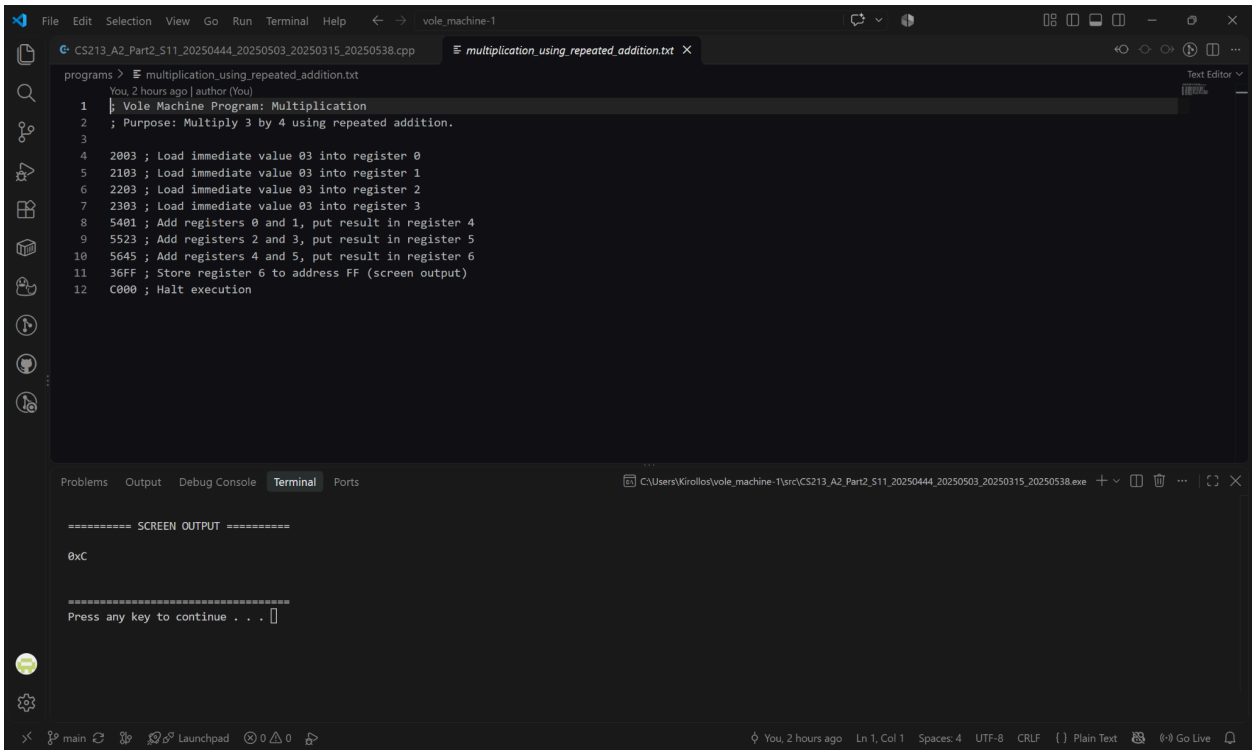
Screen 4



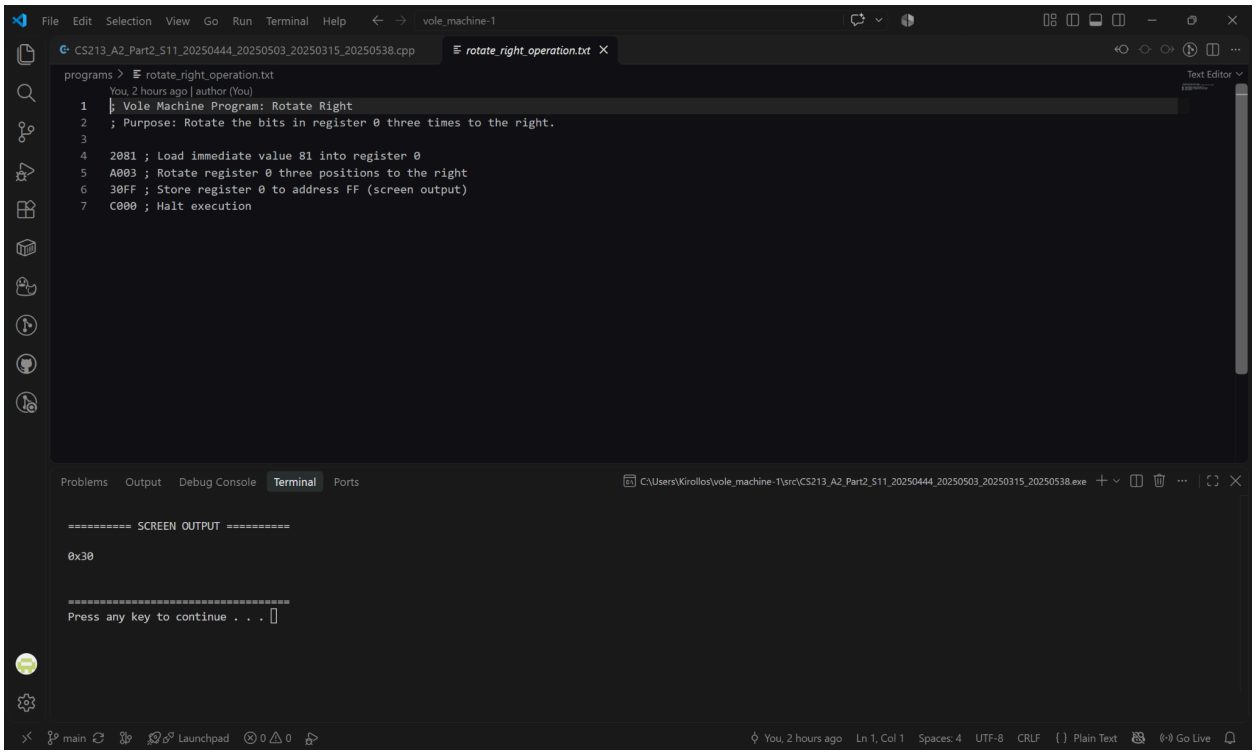
Screen 5



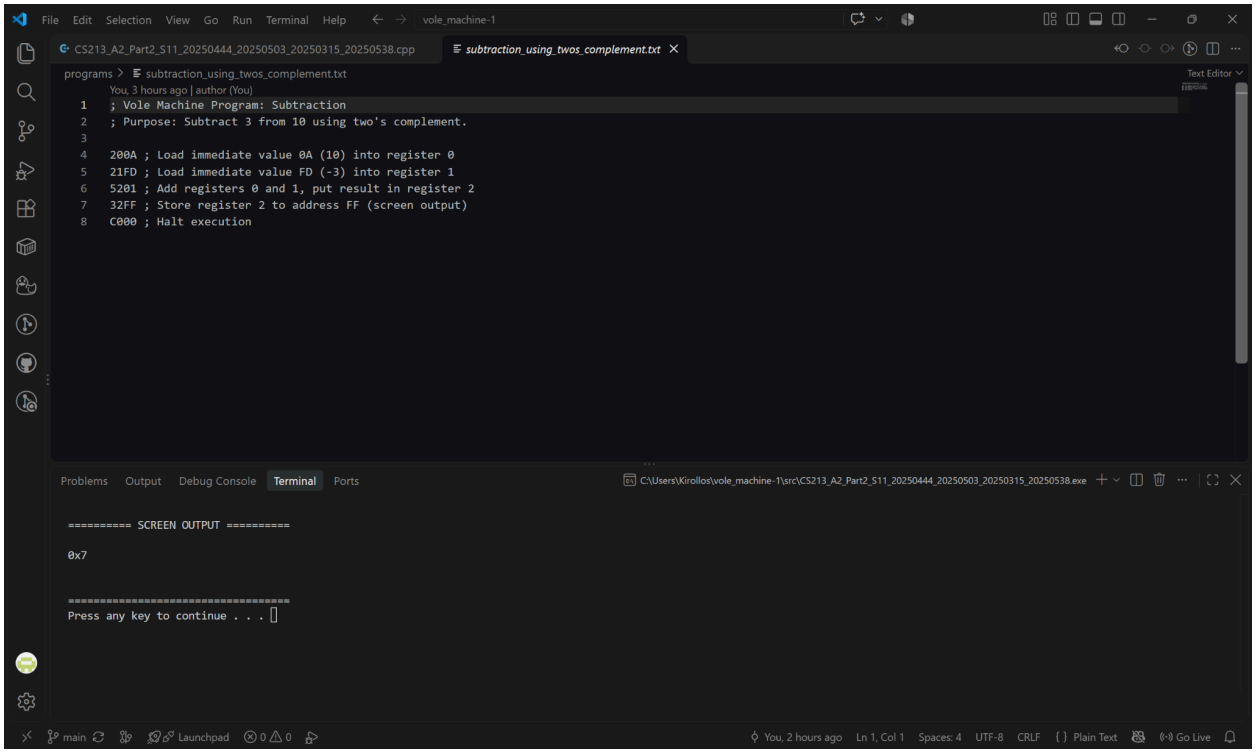
Screen 6



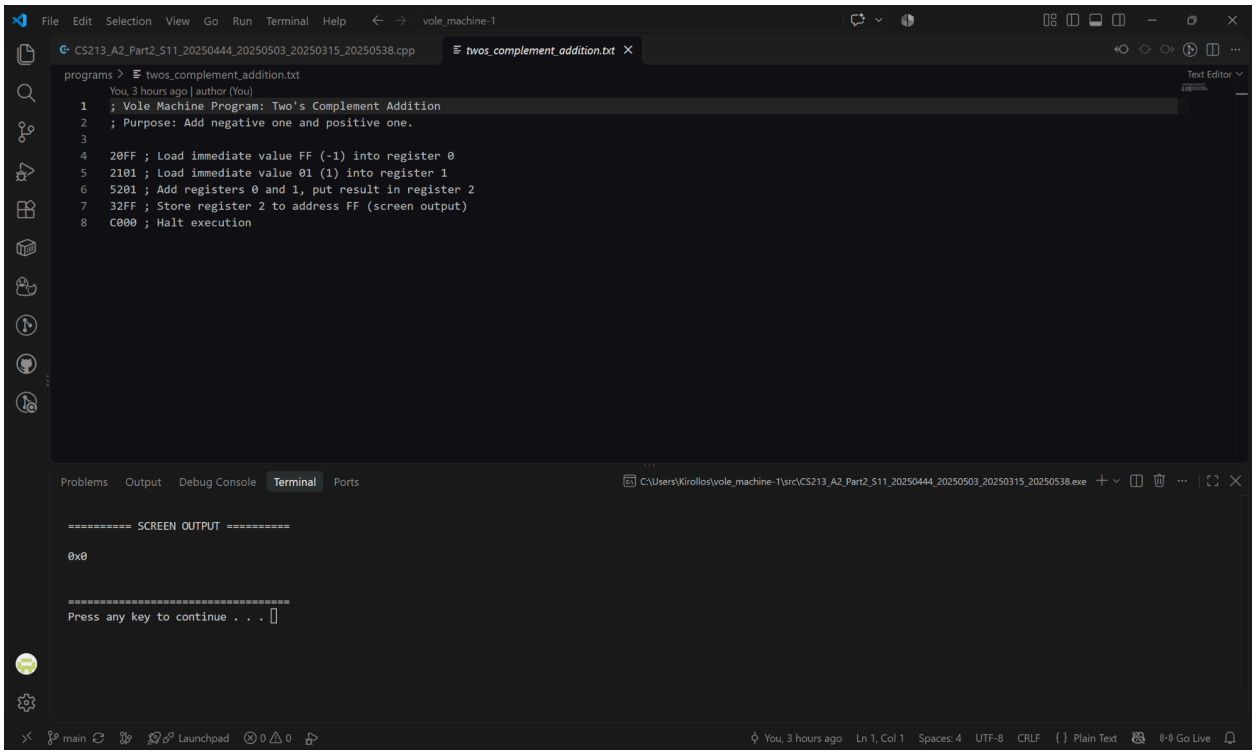
Screen 7


File Edit Selection View Go Run Terminal Help
vole_machine-1
CS213_A2_Part2_S11_20250444_20250503_20250315_20250538.cpp
rotate_right_operation.txt
programs > rotate_right_operation.txt
You, 2 hours ago | author (You)
; Vole Machine Program: Rotate Right
; Purpose: Rotate the bits in register 0 three times to the right.
2081 ; Load immediate value 81 into register 0
A003 ; Rotate register 0 three positions to the right
30FF ; Store register 0 to address FF (screen output)
C000 ; Halt execution
Text Editor
Problems Output Debug Console Terminal Ports
C:\Users\Kirollos\vole_machine-1\src\CS213_A2_Part2_S11_20250444_20250503_20250315_20250538.exe
========== SCREEN OUTPUT ==========
0x30
==================================
Press any key to continue . . .
main Launchpad 0 0
You, 2 hours ago Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Plain Text Go Live

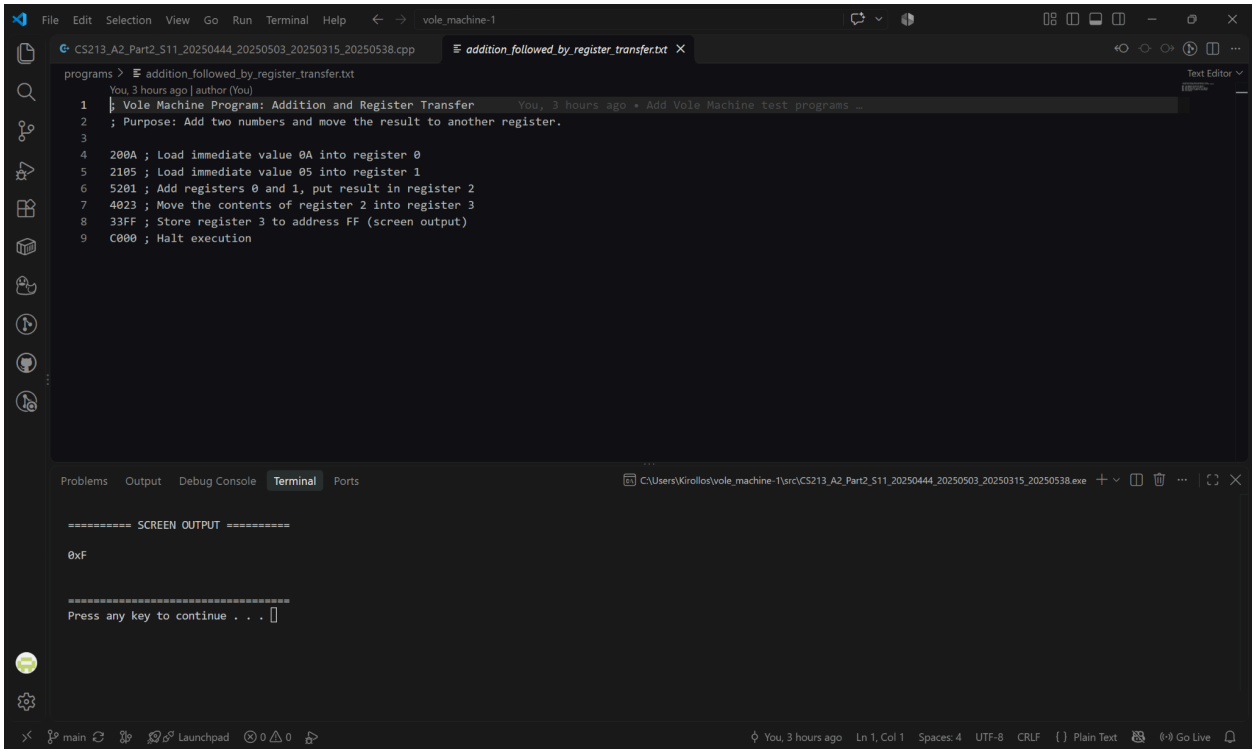
Screen 8



Screen 9



Screen 10


File Edit Selection View Go Run Terminal Help
vole_machine-1
CS213_A2_Part2_S11_20250444_20250503_20250315_20250538.cpp
addition_followed_by_register_transfer.txt
programs > addition_followed_by_register_transfer.txt
Text Editor
You, 3 hours ago | author (You)
; Vole Machine Program: Addition and Register Transfer
You, 3 hours ago • Add Vole Machine test programs …
; Purpose: Add two numbers and move the result to another register.
200A ; Load immediate value 0A into register 0
2105 ; Load immediate value 05 into register 1
5201 ; Add registers 0 and 1, put result in register 2
4023 ; Move the contents of register 2 into register 3
33FF ; Store register 3 to address FF (screen output)
C000 ; Halt execution
Problems Output Debug Console Terminal Ports
C:\Users\Kirollos\vole_machine-1\src\CS213_A2_Part2_S11_20250444_20250503_20250315_20250538.exe
========== SCREEN OUTPUT ==========
0xF
==================================
Press any key to continue . . .
main Launchpad 0 0
You, 3 hours ago Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Plain Text Go Live